

EECS 498/CSE 598: Zero-Knowledge Proofs

Winter 2026

Lecture 21

Paul Grubbs

paulgrub@umich.edu

Beyster 4709

Agenda for this lecture

- Announcements
- Zero-knowledge middleboxes (motivation)
- Zero-knowledge middleboxes for DNS filtering
- zkTLS: the problem it solves

Agenda for this lecture

- **Announcements**
- Zero-knowledge middleboxes (motivation)
- Zero-knowledge middleboxes for DNS filtering
- zkTLS: the problem it solves

Announcements

- P4 and theory pset 2 are out!.
 - Note there will be no P5 nor a final exam. These will be the last assignments of the semester.
- All work for this course must be completed by our final exam date, April 27th
- Reminder: you can get participation credit for going to discussion.
 - Chad is getting lonely on Fridays!
- **PLEASE** do your course evaluations – they are really crucial for getting feedback on this class!!!

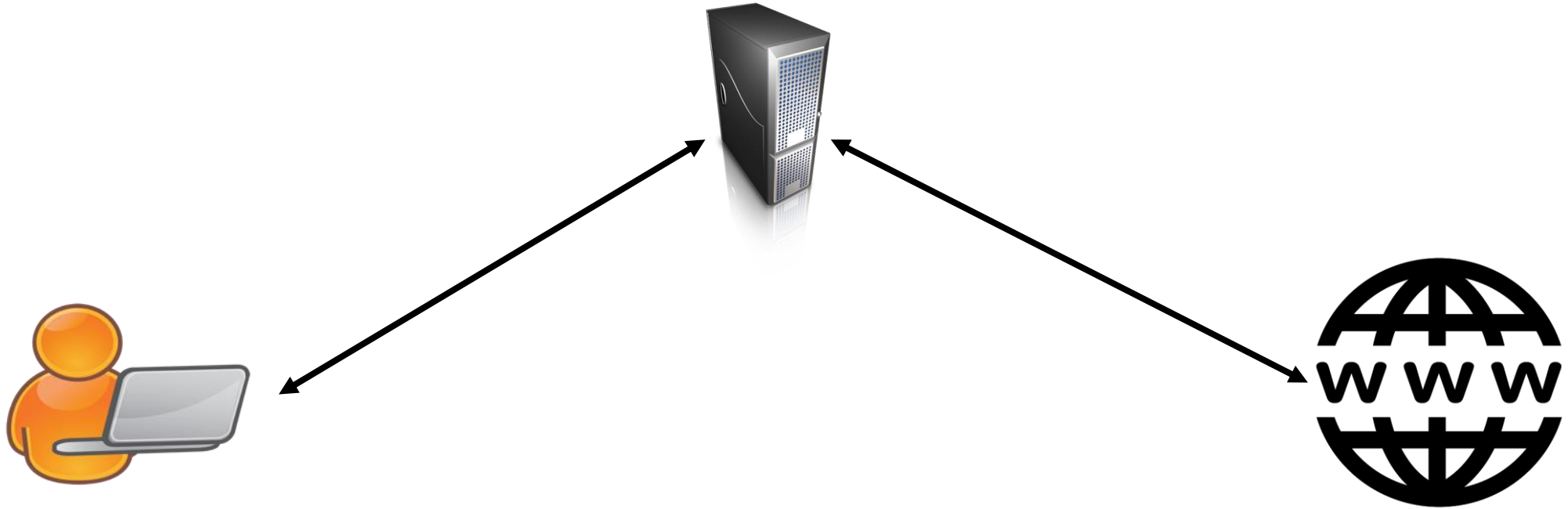


Agenda for this lecture

- Announcements
- Zero-knowledge middleboxes (motivation)
- Zero-knowledge middleboxes for DNS filtering
- zkTLS: the problem it solves

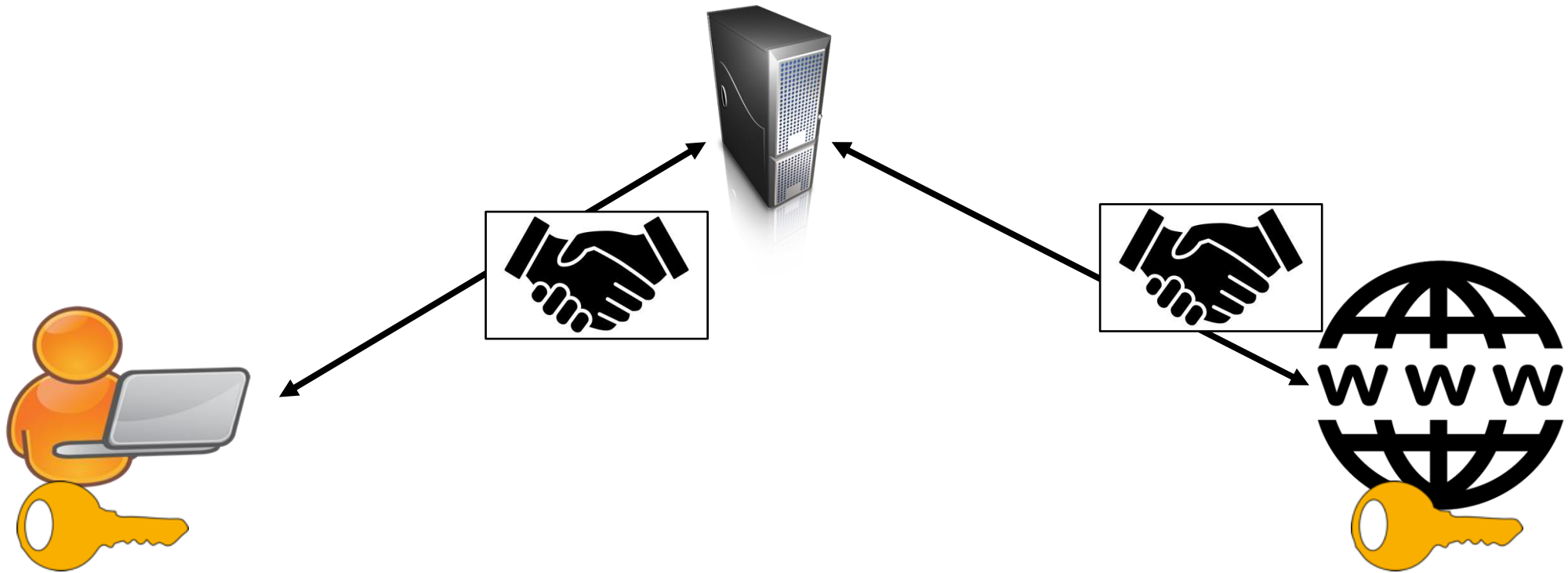
Privacy via Encryption

Encryption hides data;
increasingly hides
metadata too.



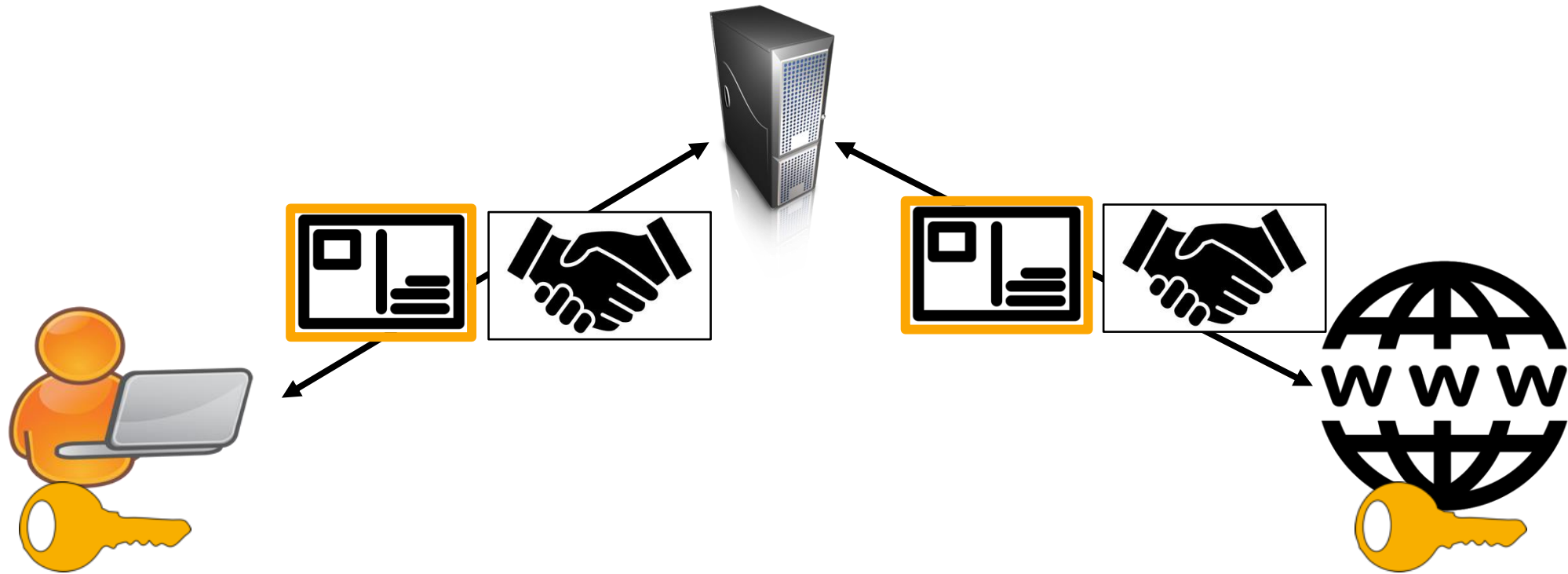
Privacy via Encryption

Encryption hides data;
increasingly hides
metadata too.



Privacy via Encryption

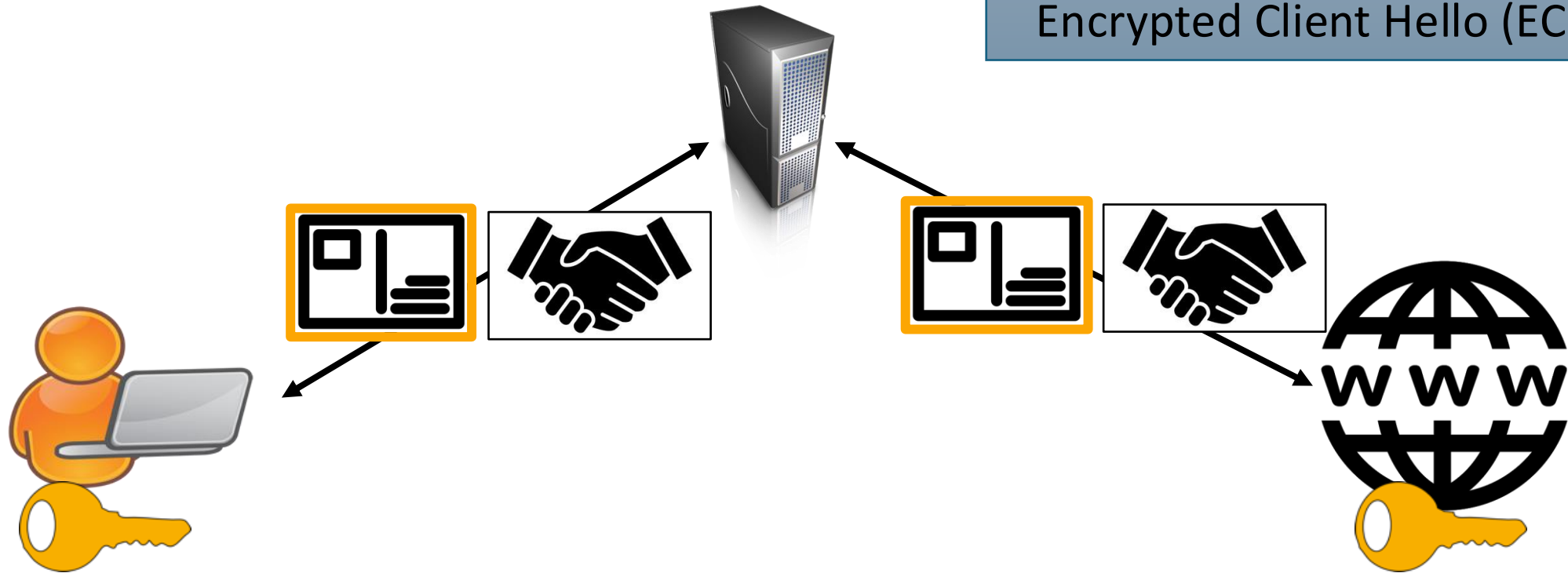
Encryption hides data;
increasingly hides
metadata too.



Privacy via Encryption

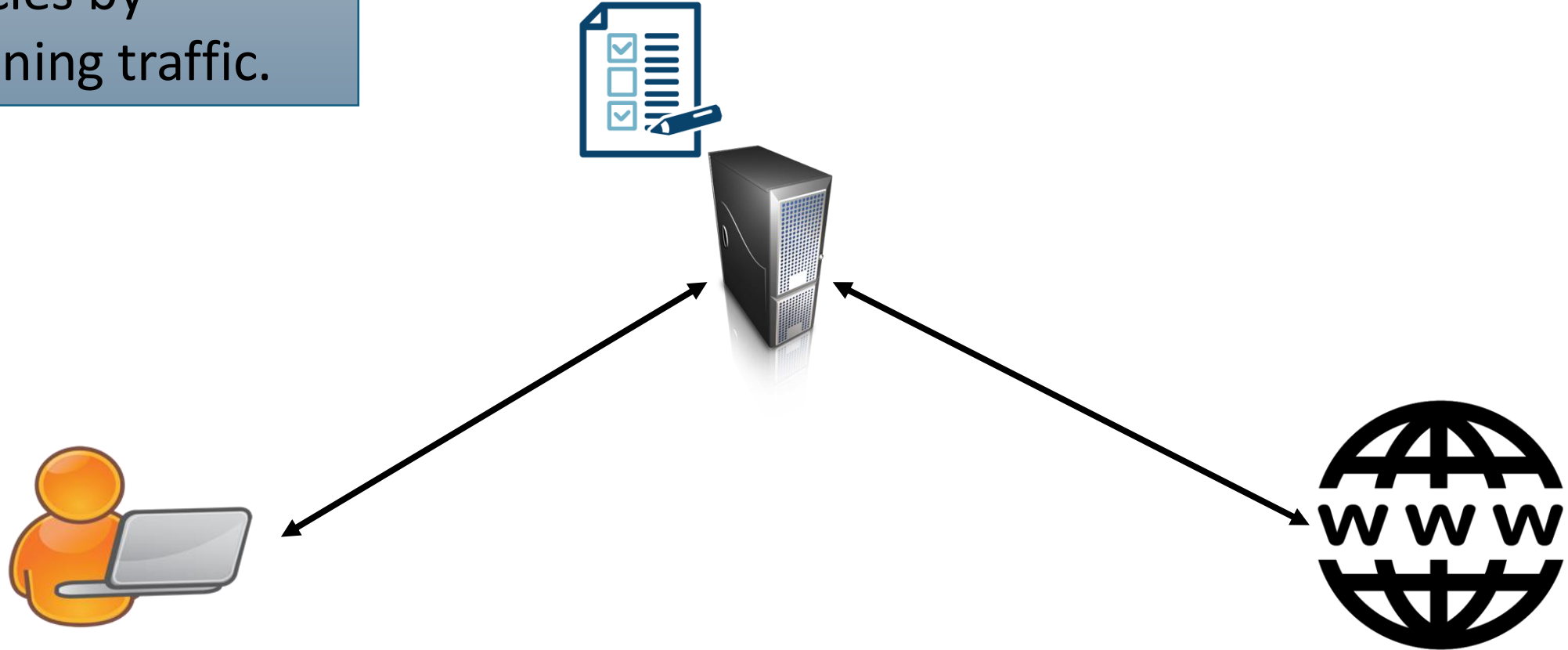
Encryption hides data;
increasingly hides
metadata too.

Encrypted DNS (DoH/DoT)
Oblivious DoH
TLS 1.3:
Encrypted certificates
Encrypted Client Hello (ECH)



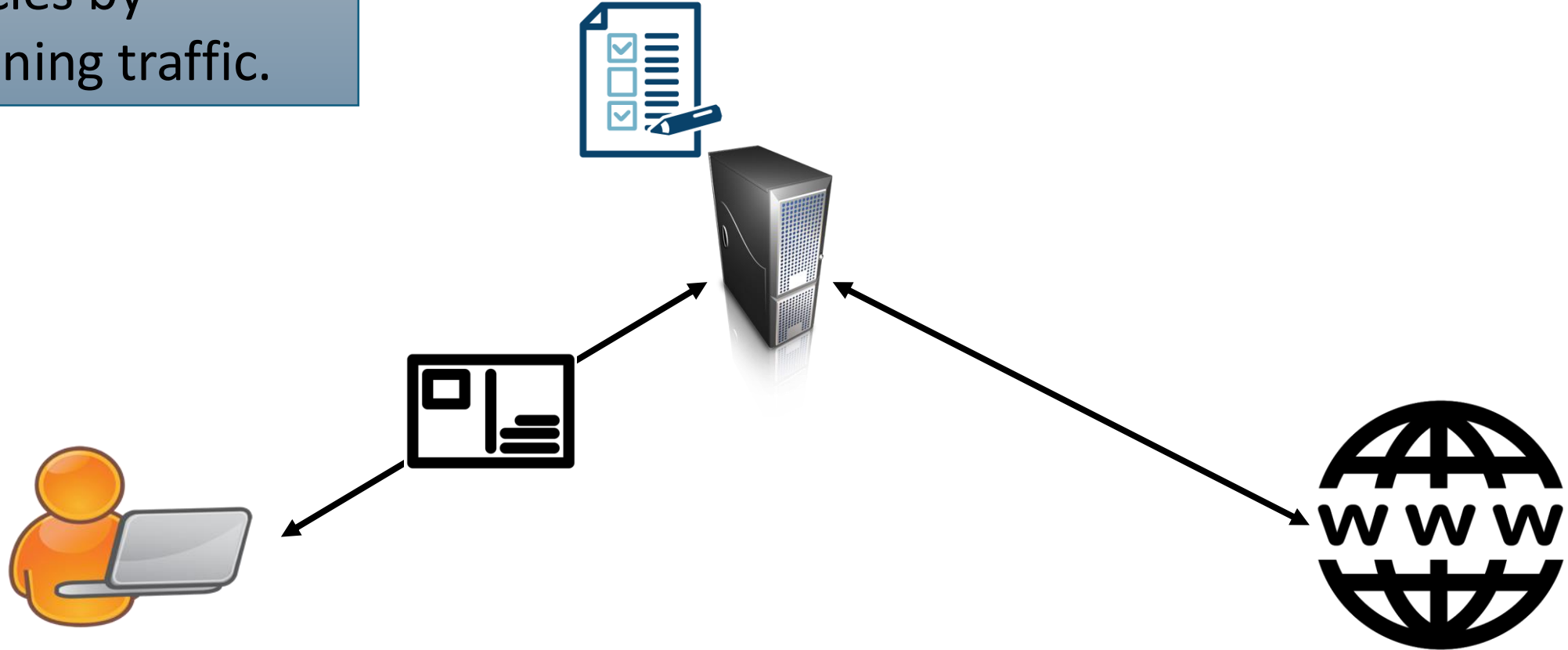
Policy Enforcement

Networks enforce policies by scanning traffic.



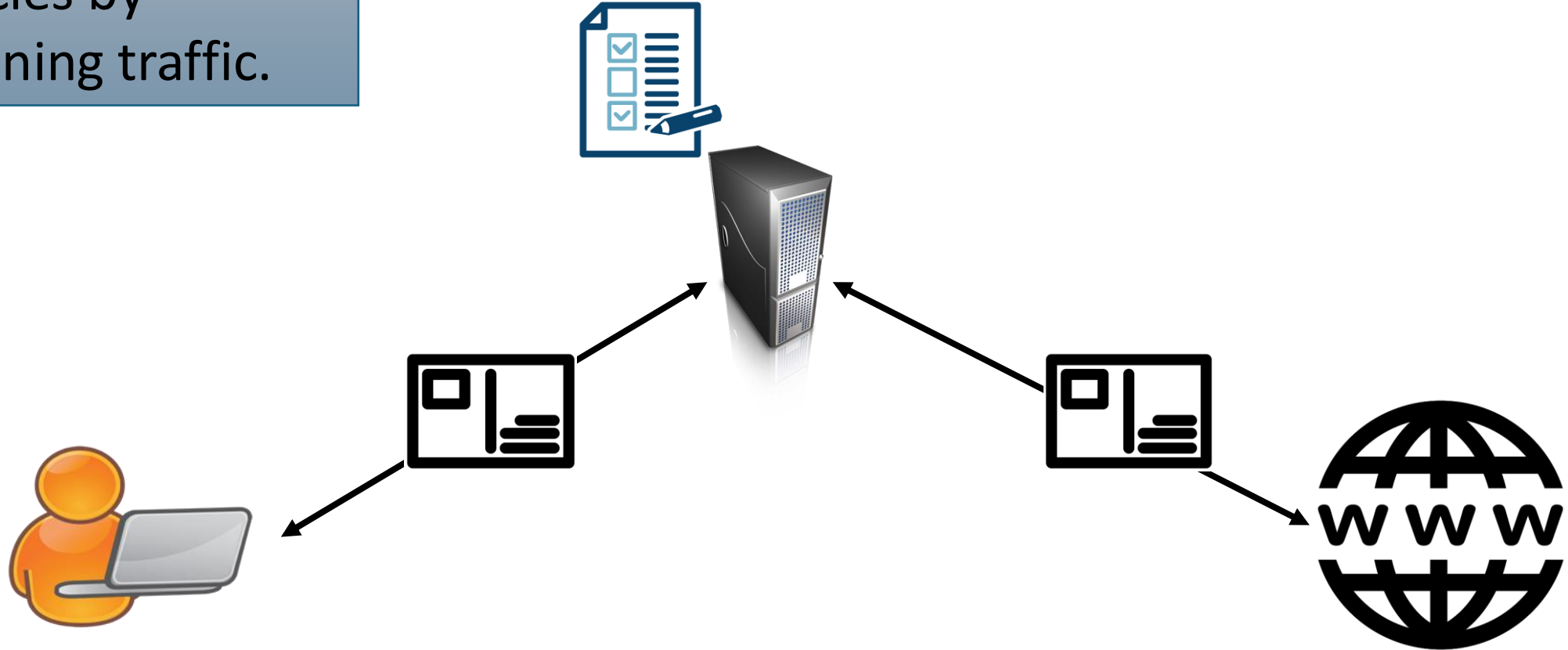
Policy Enforcement

Networks enforce policies by scanning traffic.



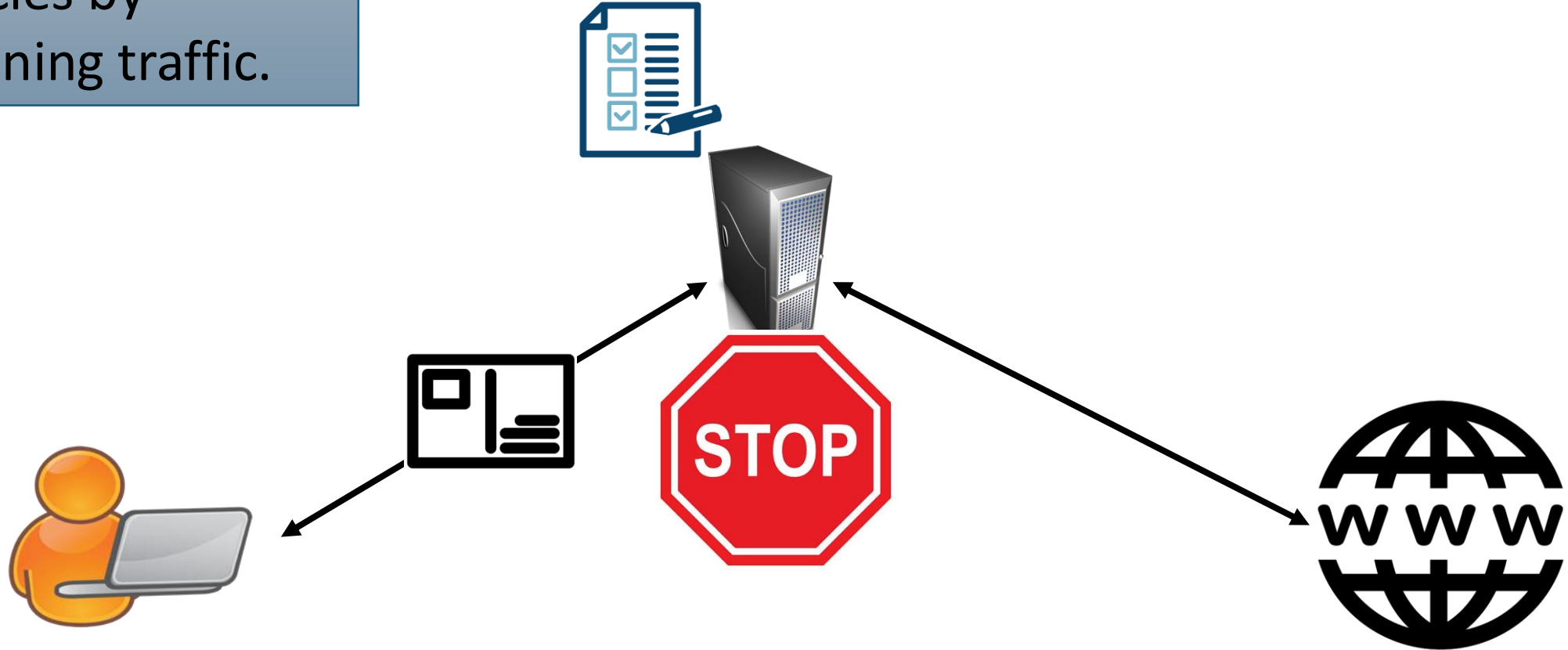
Policy Enforcement

Networks enforce policies by scanning traffic.



Policy Enforcement

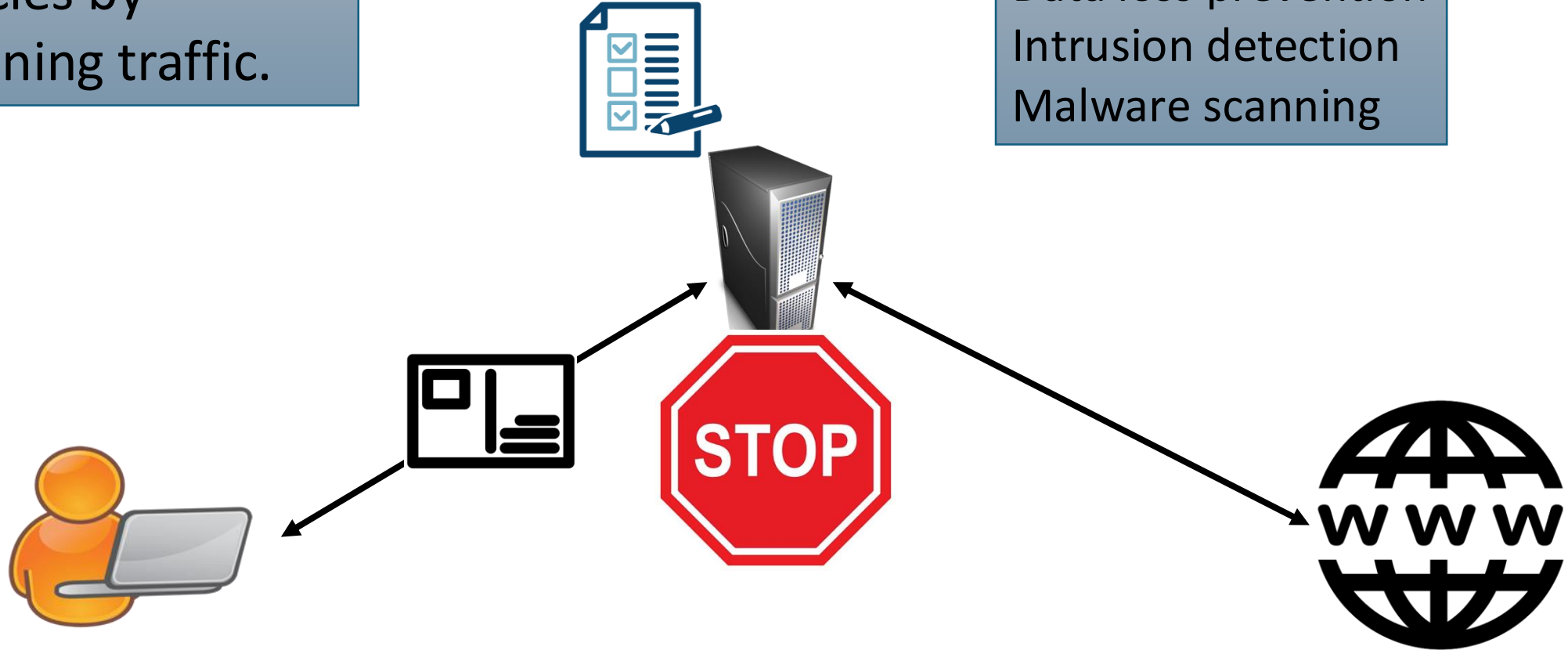
Networks enforce policies by scanning traffic.



Policy Enforcement

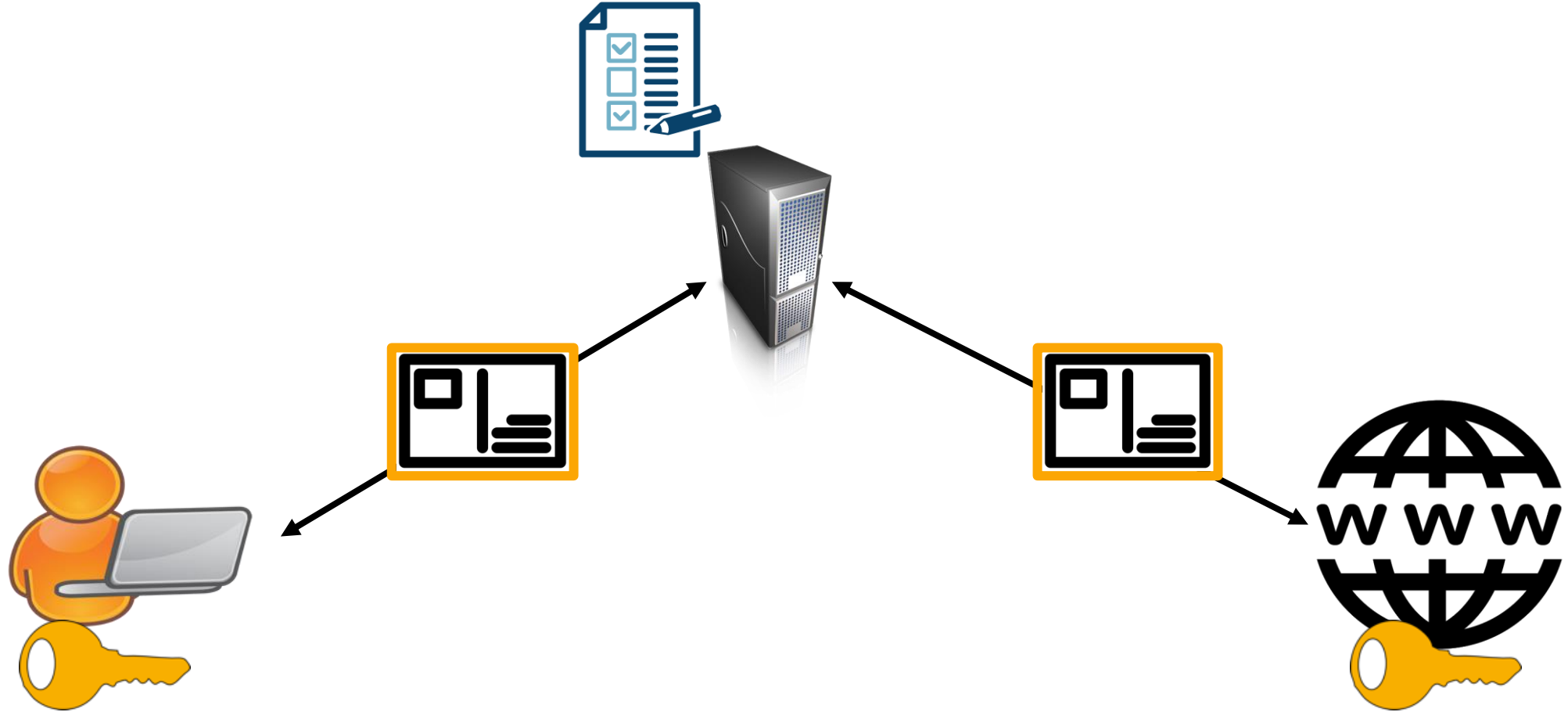
Networks enforce policies by scanning traffic.

DNS filtering
Data loss prevention
Intrusion detection
Malware scanning

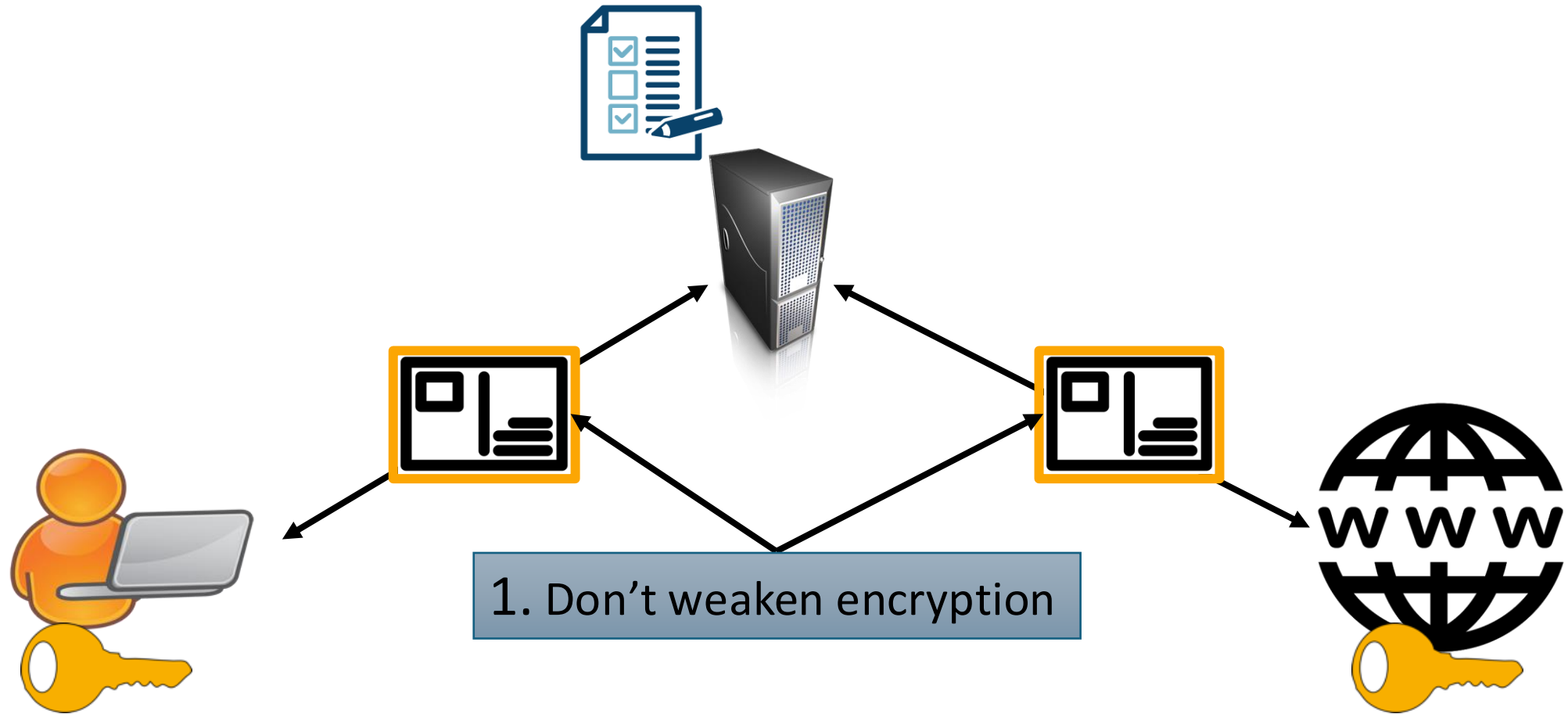


Is it possible to have both privacy
and policy enforcement?

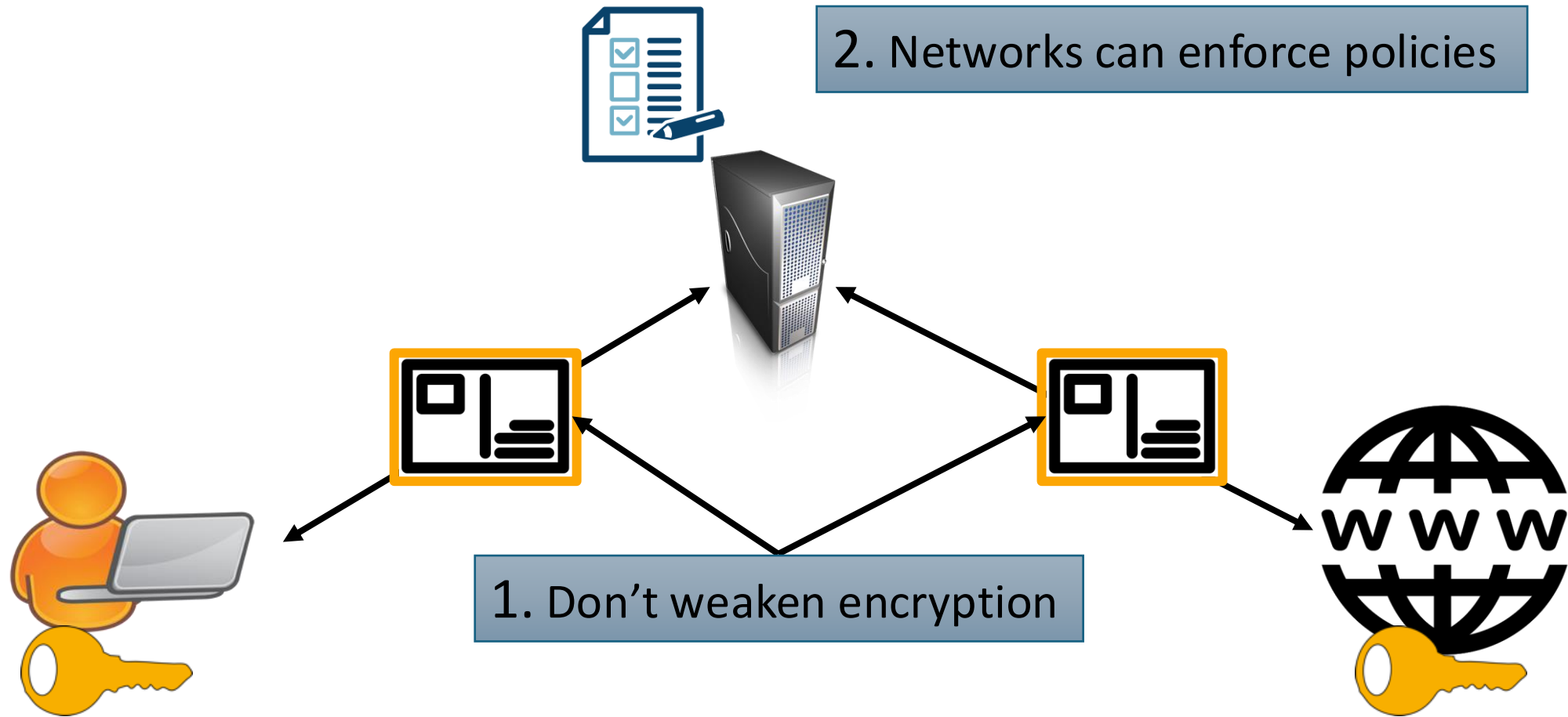
Requirements



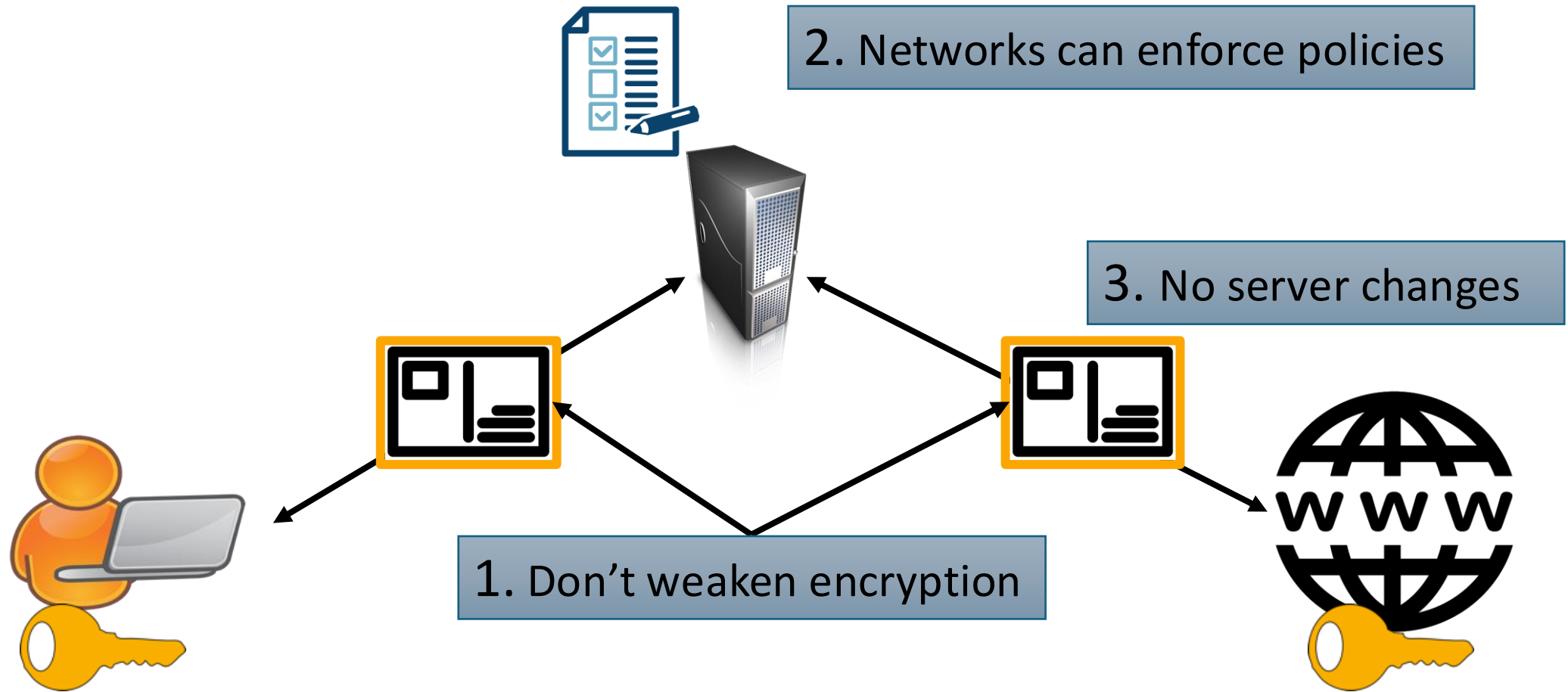
Requirements



Requirements



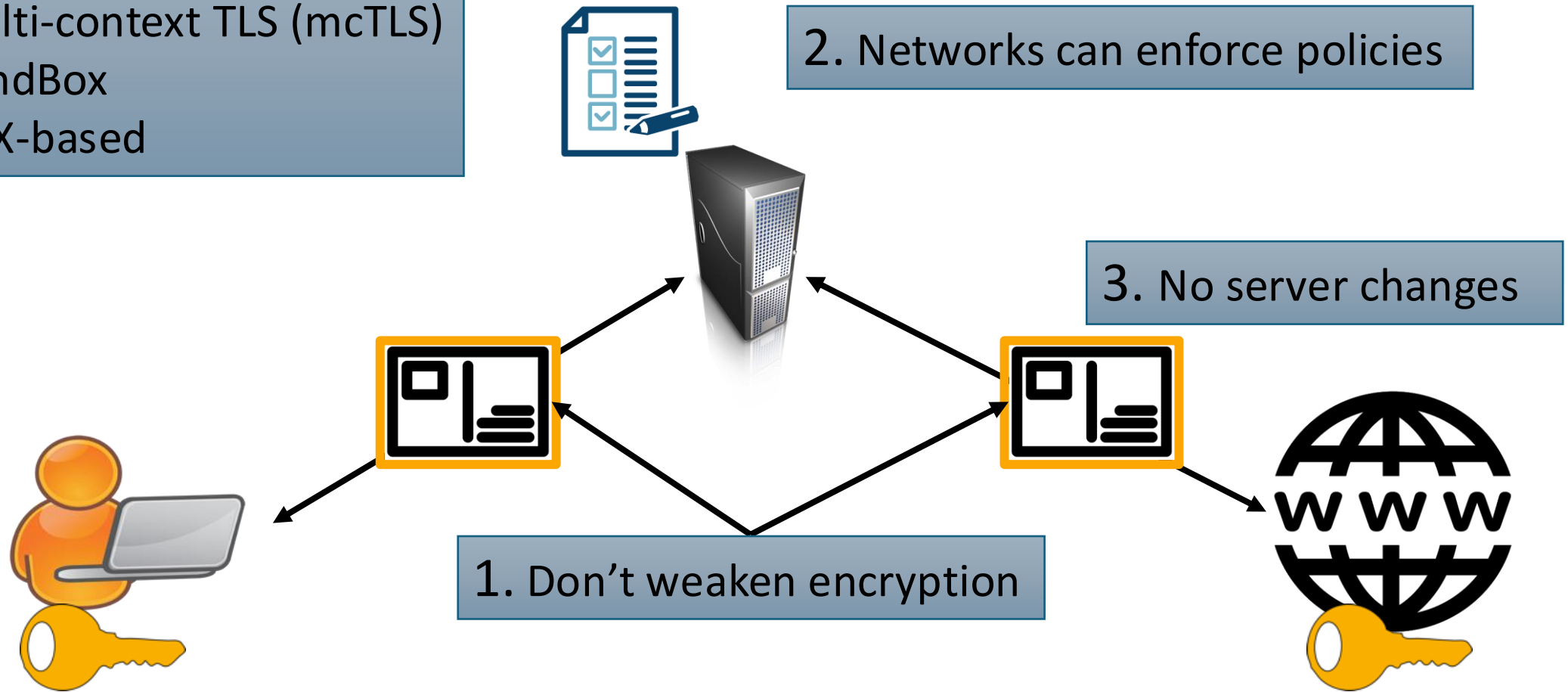
Requirements



Requirements

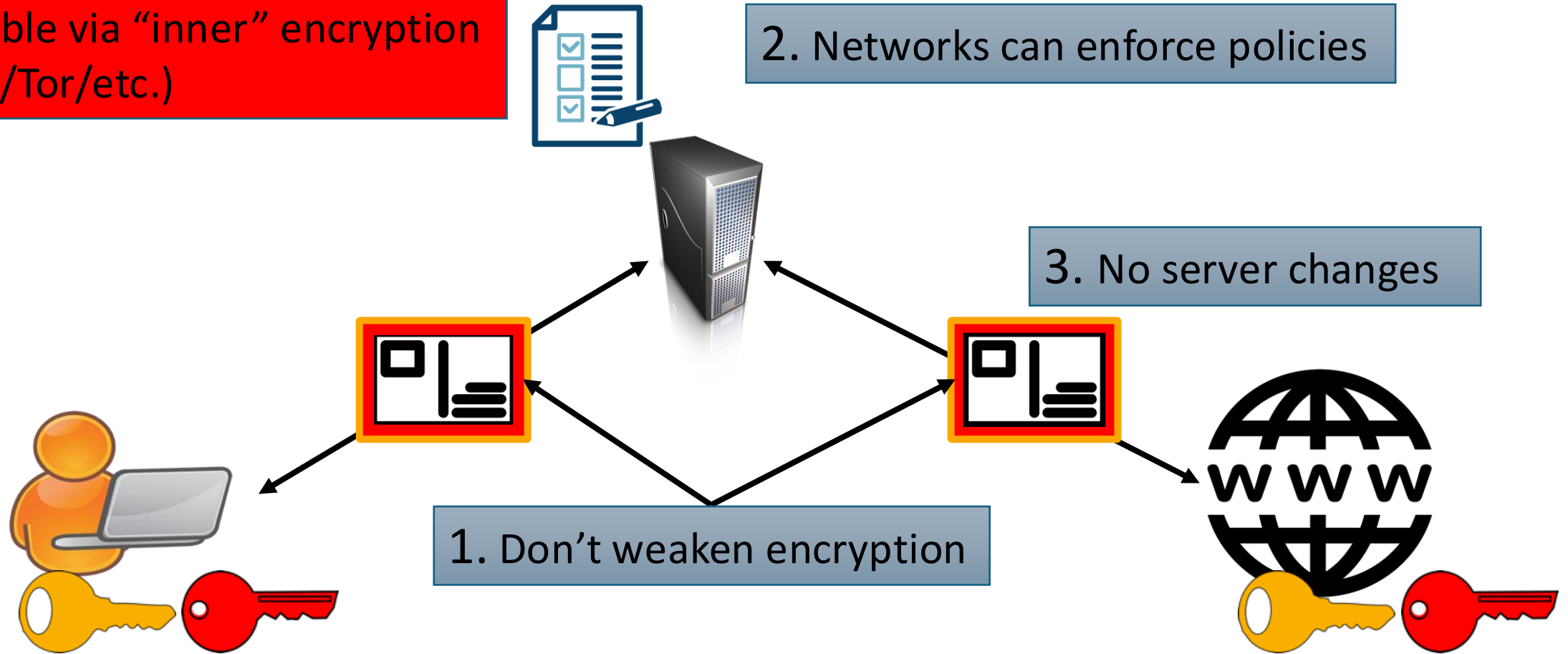
Prior work:

- multi-context TLS (mcTLS)
- BlindBox
- SGX-based

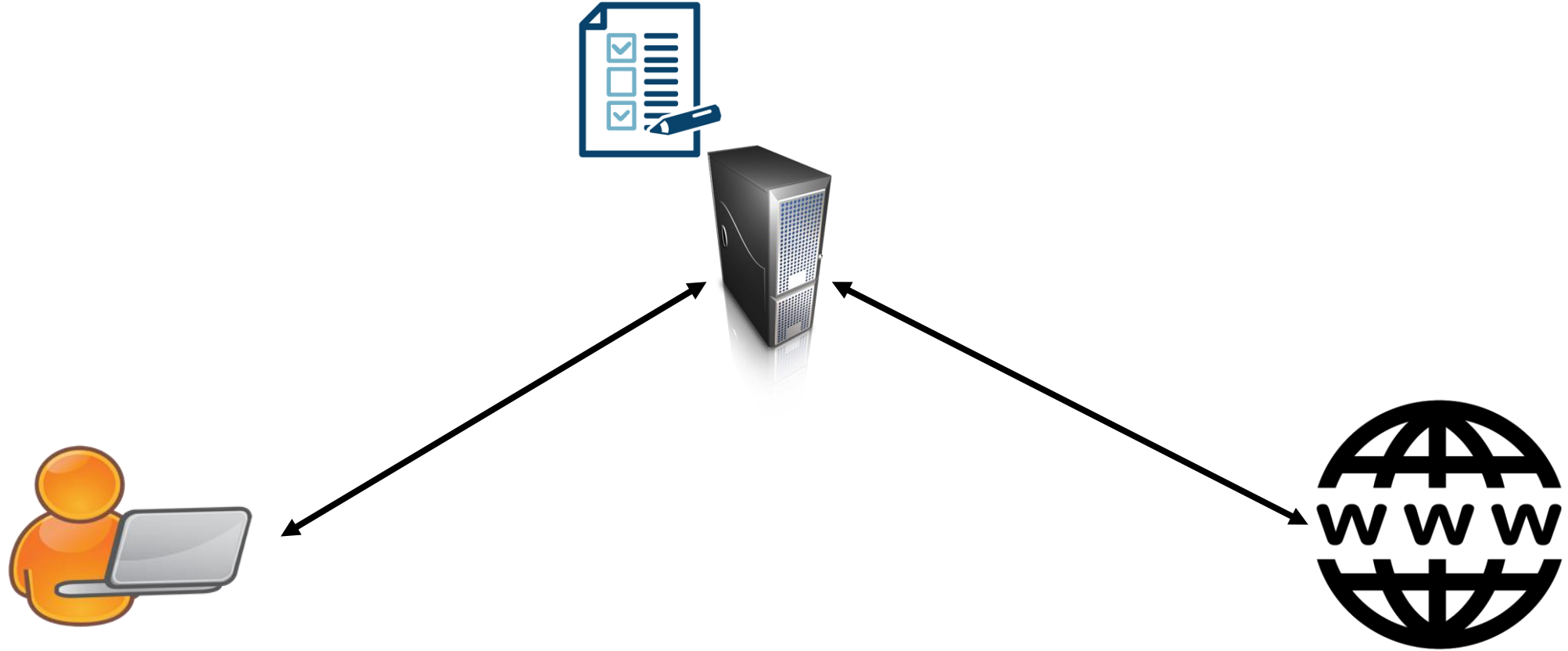


Circumvention

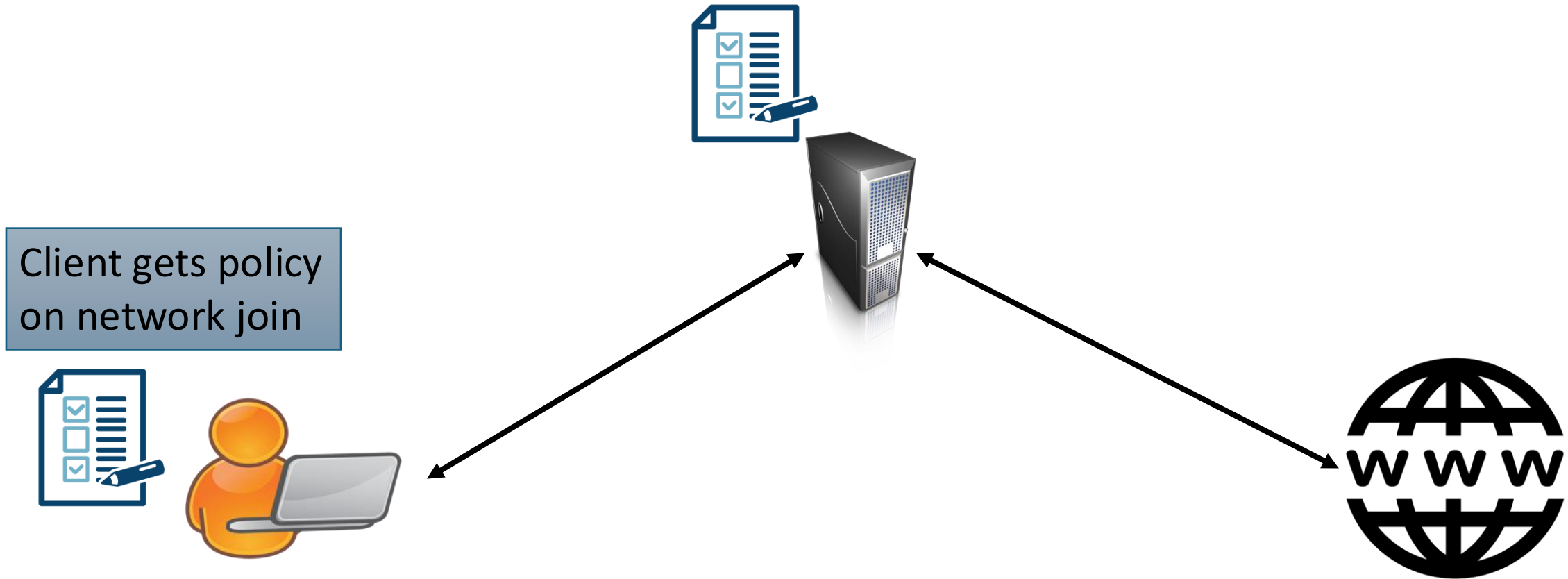
Circumvention should still be possible via “inner” encryption (VPN/Tor/etc.)



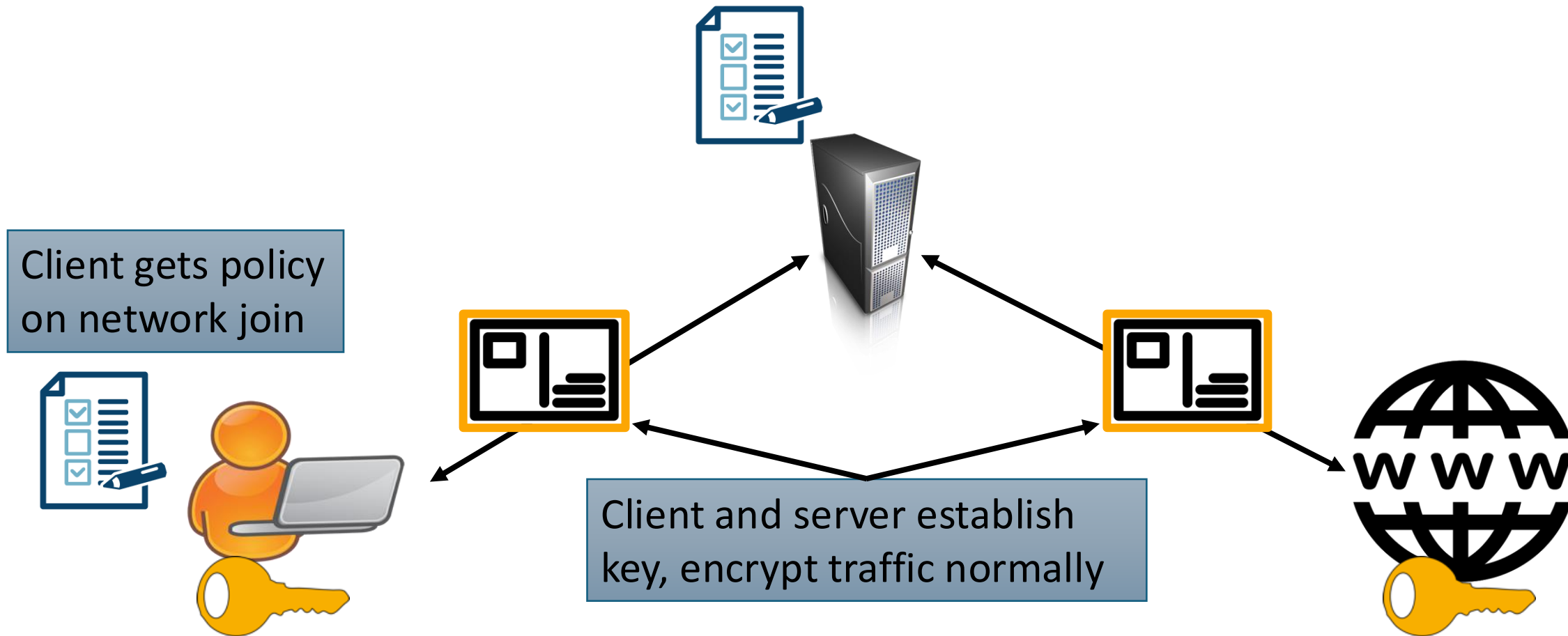
Zero-Knowledge Middleboxes



Zero-Knowledge Middleboxes



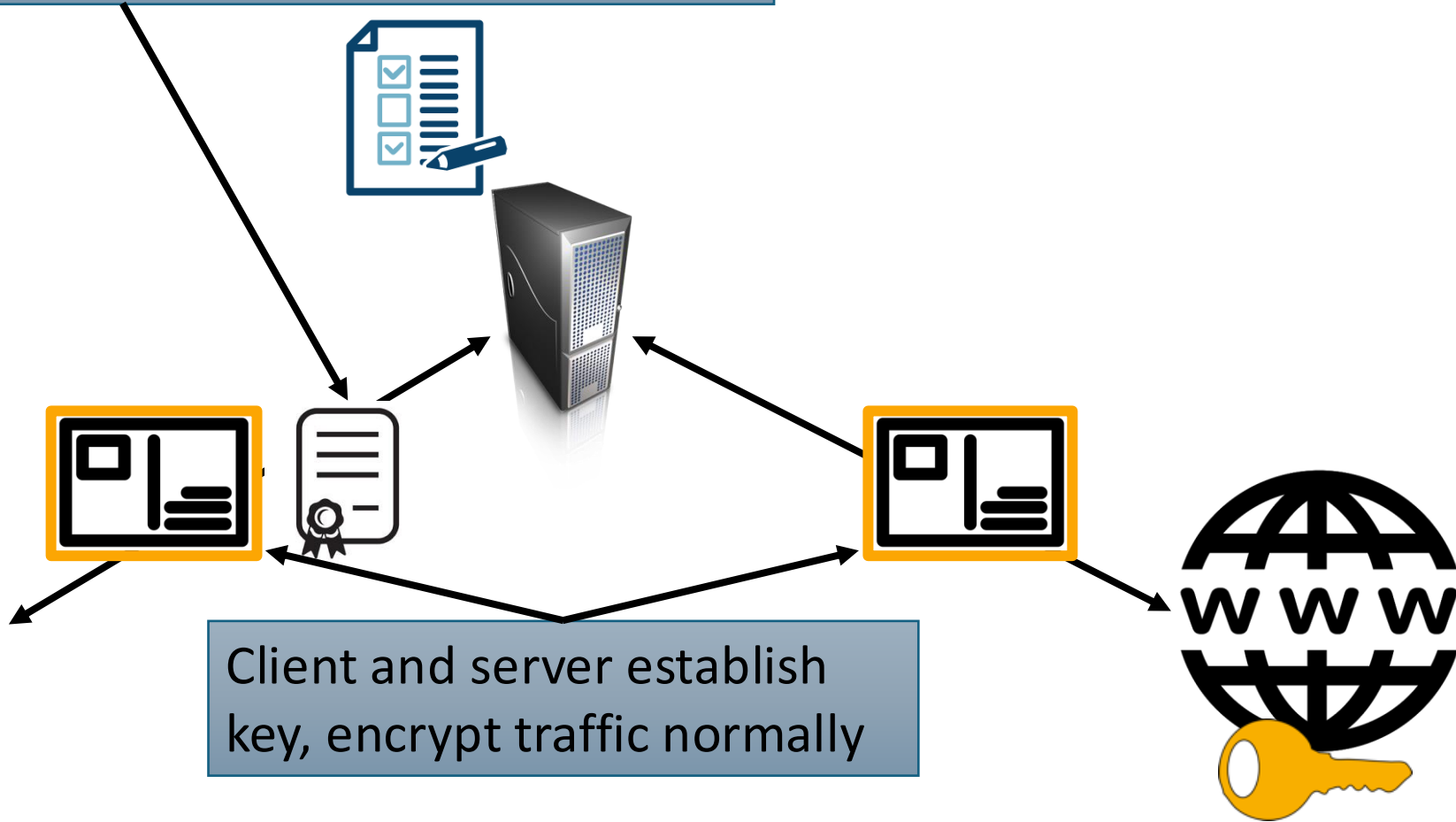
Zero-Knowledge Middleboxes



Zero-Knowledge Middleboxes

Client enforces policies locally, sends ZKP for statement
“My ciphertext contains compliant traffic”

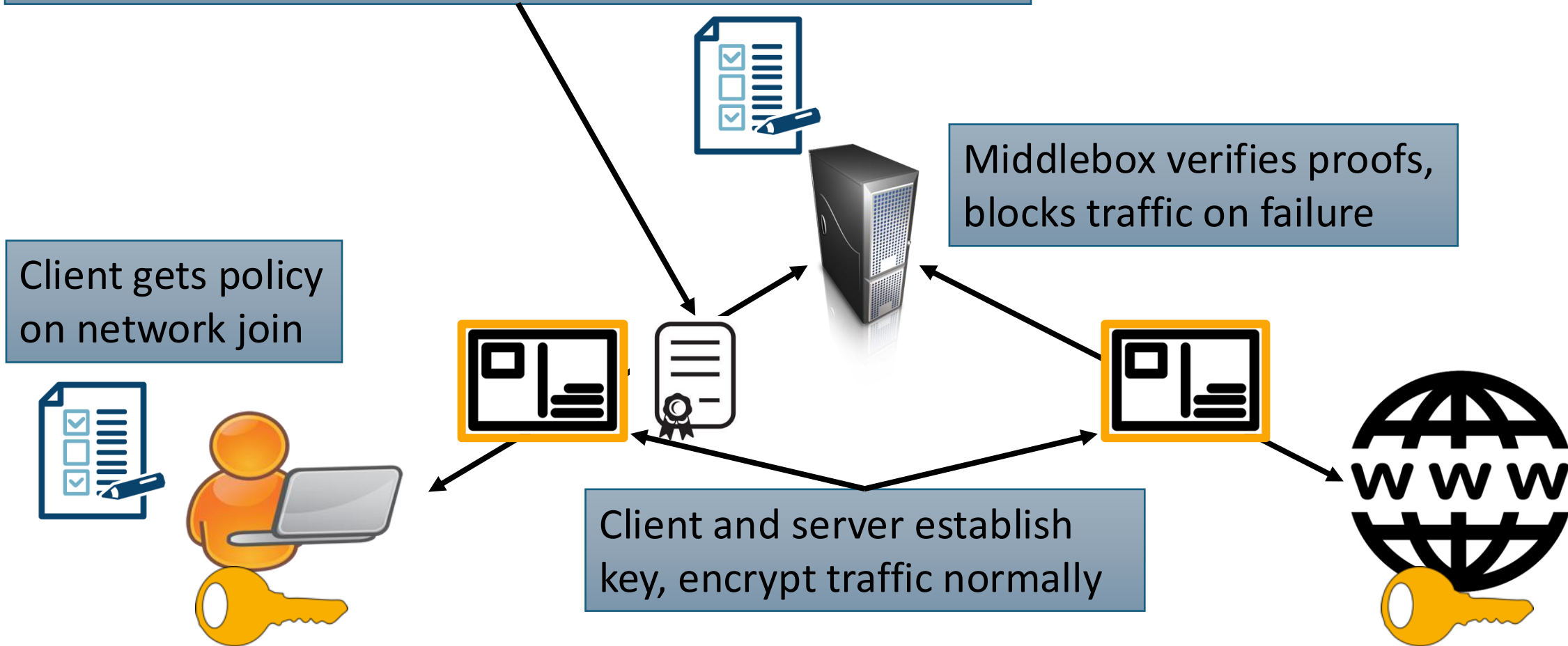
Client gets policy
on network join



Client and server establish
key, encrypt traffic normally

Zero-Knowledge Middleboxes

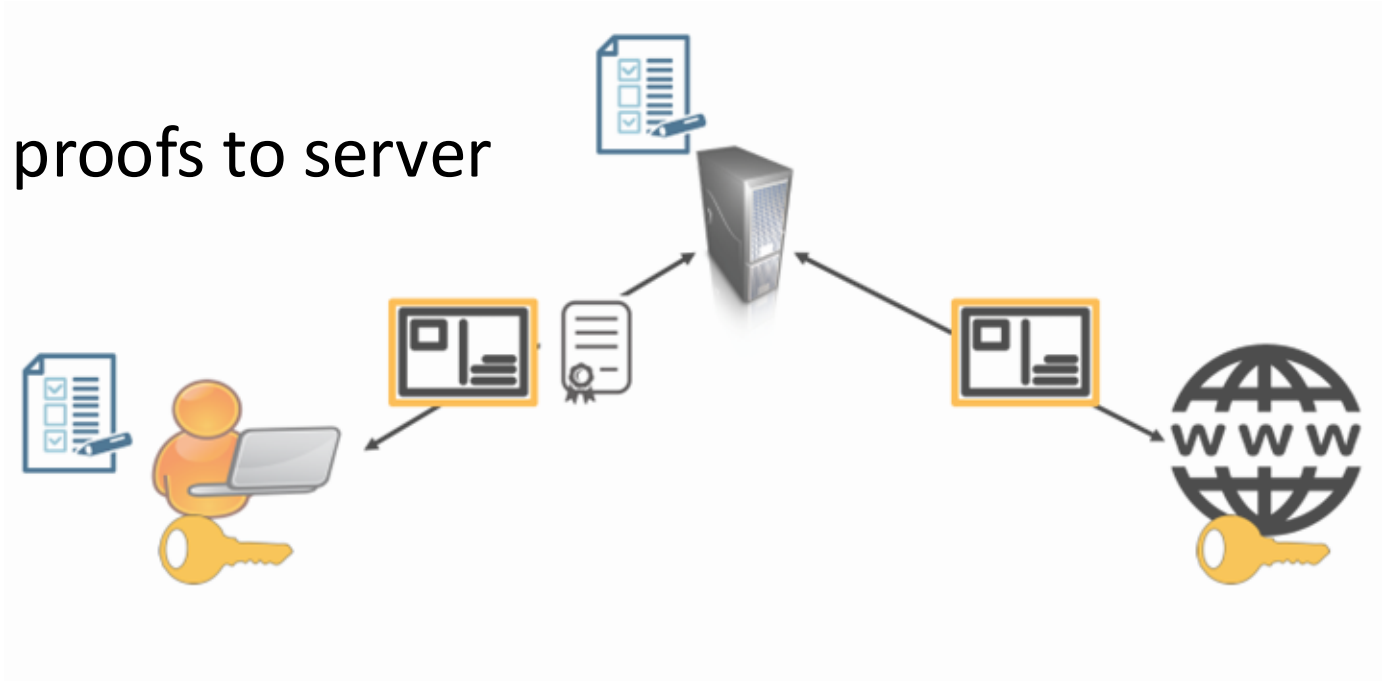
Client enforces policies locally, sends ZKP for statement
“My ciphertext contains compliant traffic”

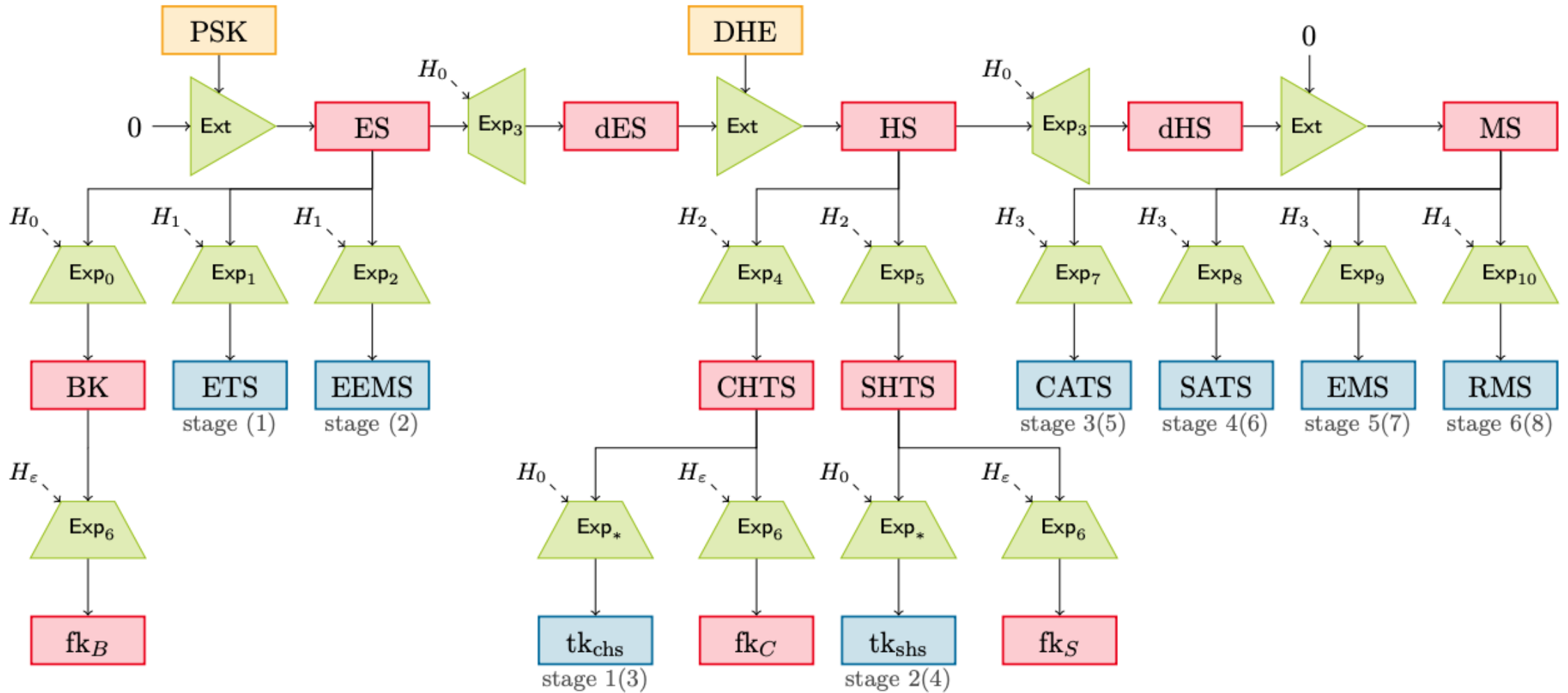


Zero-Knowledge Middleboxes

Requirements:

1. Don't weaken encryption
 - ✓ Using standard encryption + zero-knowledge property of ZKP
2. Middlebox can enforce policies
 - ✓ ZKP soundness
3. No server changes
 - ✓ Middlebox doesn't forward proofs to server



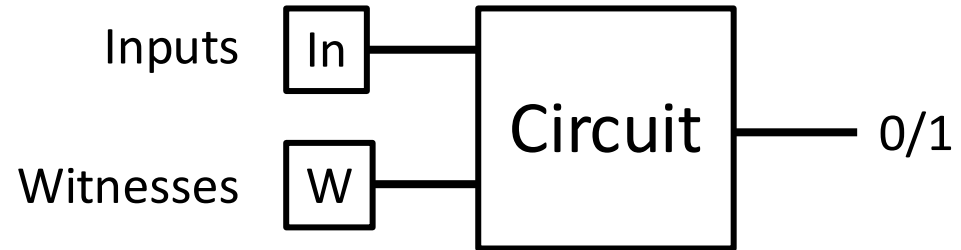


“A Cryptographic Analysis of the TLS 1.3 Handshake Protocol” – Dowling et al.

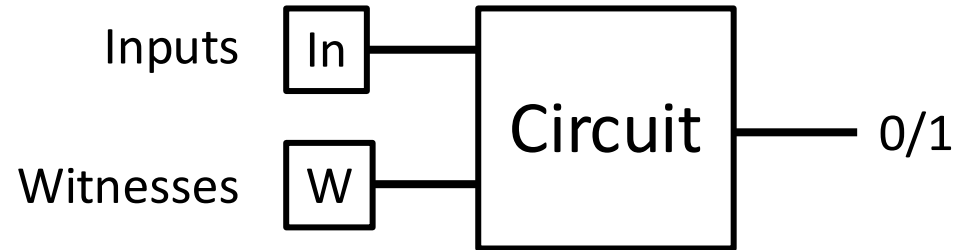
Agenda for this lecture

- Announcements
- Zero-knowledge middleboxes (motivation)
- Zero-knowledge middleboxes for DNS filtering
- zkTLS: the problem it solves

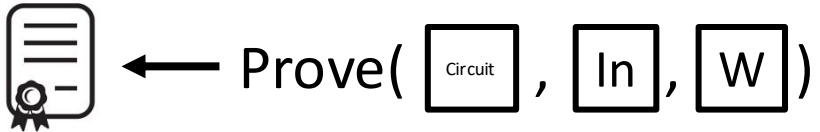
Circuits for ZKPs



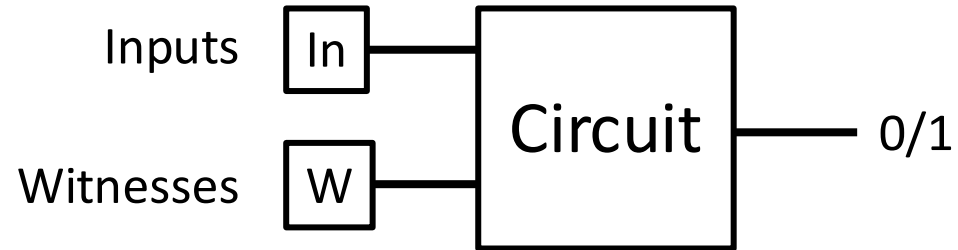
Circuits for ZKPs



Prover



Circuits for ZKPs



Prover



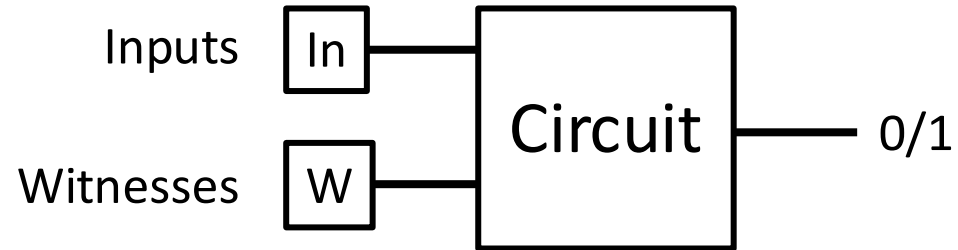
← Prove(Circuit , In , W)



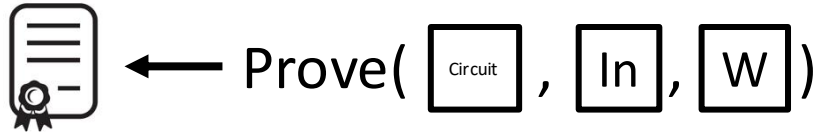
Verifier



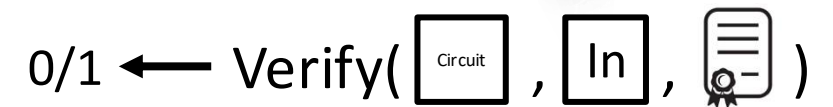
Circuits for ZKPs



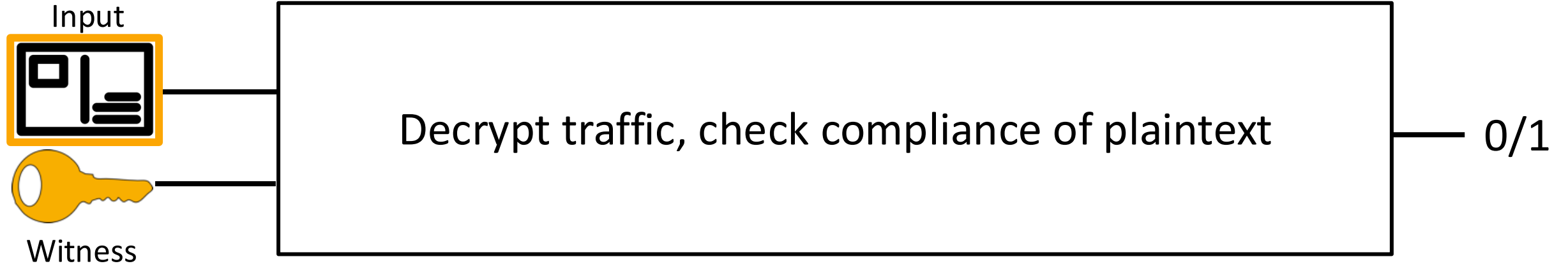
Prover



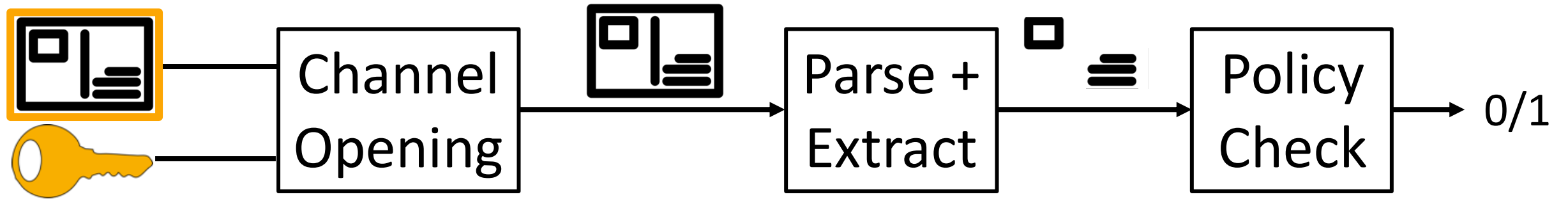
Verifier



ZKMB Circuits



ZKMB Circuits



Function

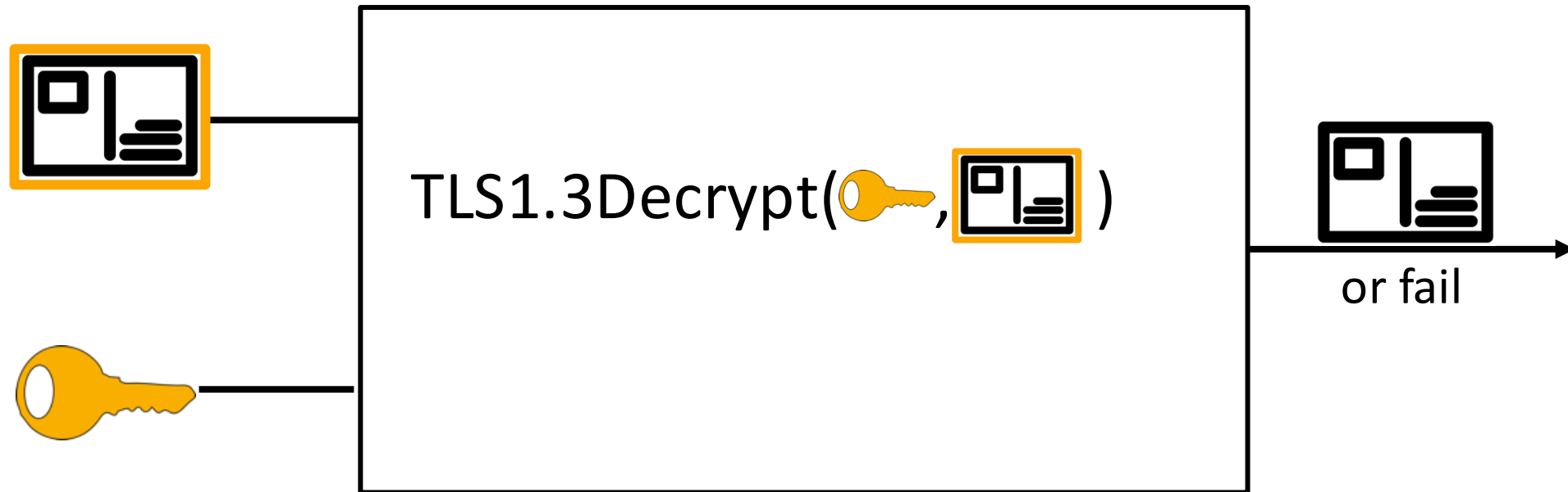
Decrypts ciphertext,
outputs message

Finds, outputs relevant
data from message

Verifies data is
compliant

Channel Opening for TLS 1.3

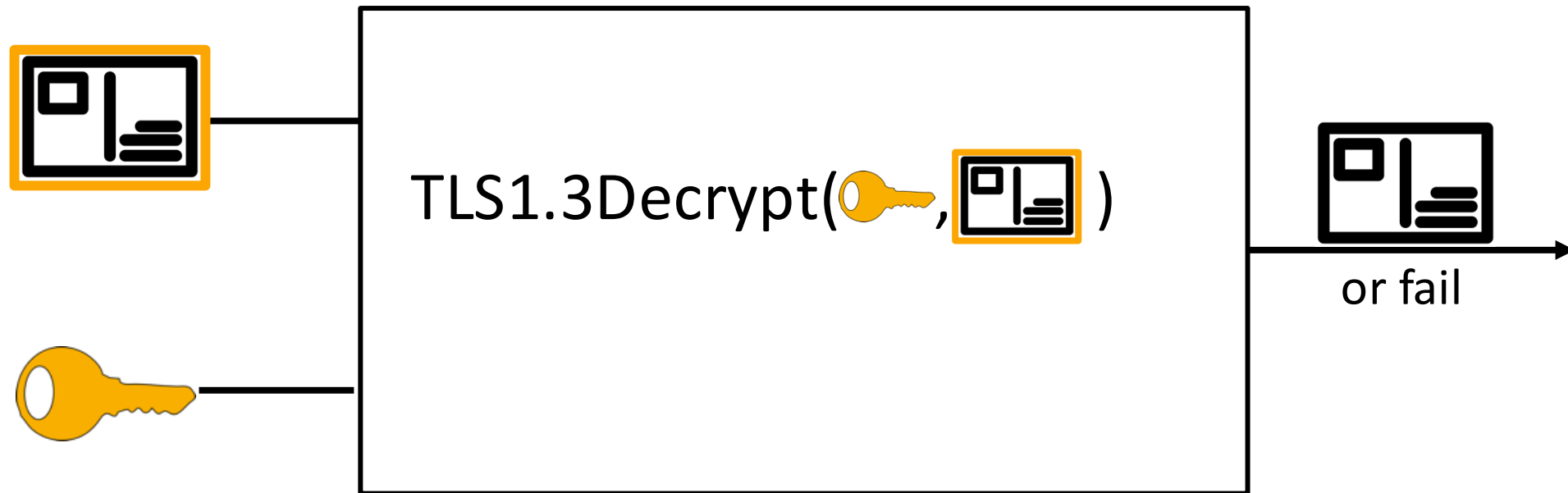
How to open a TLS 1.3 ciphertext?



Channel Opening for TLS 1.3

How to open a TLS 1.3 ciphertext?

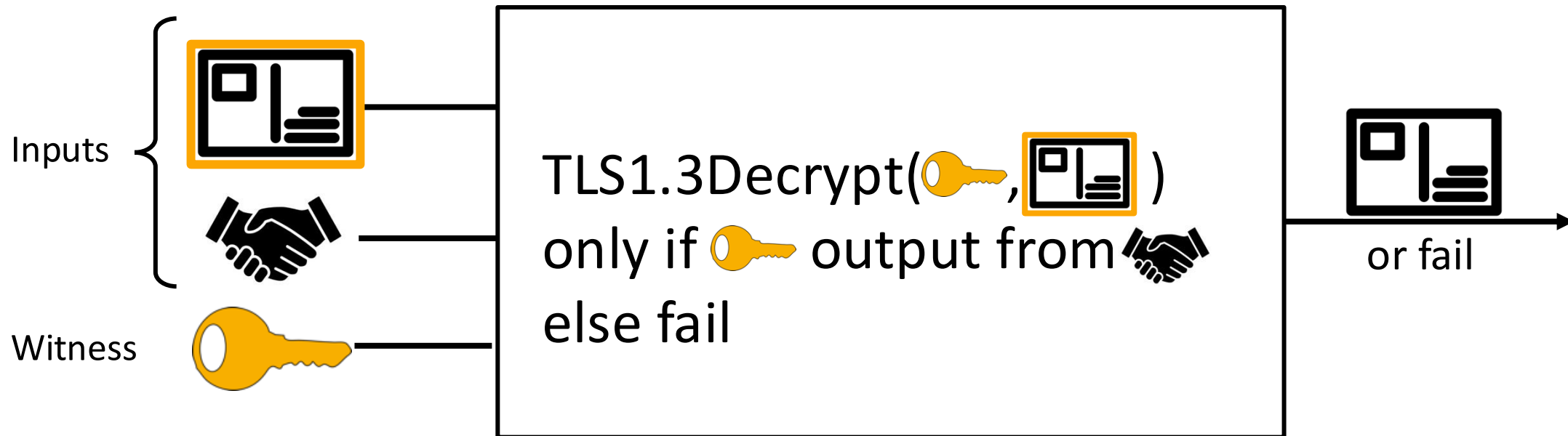
Problem: TLS 1.3 AEADs are not binding:
ciphertexts have multiple correct decryptions.



Channel Opening for TLS 1.3

How to open a TLS 1.3 ciphertext?

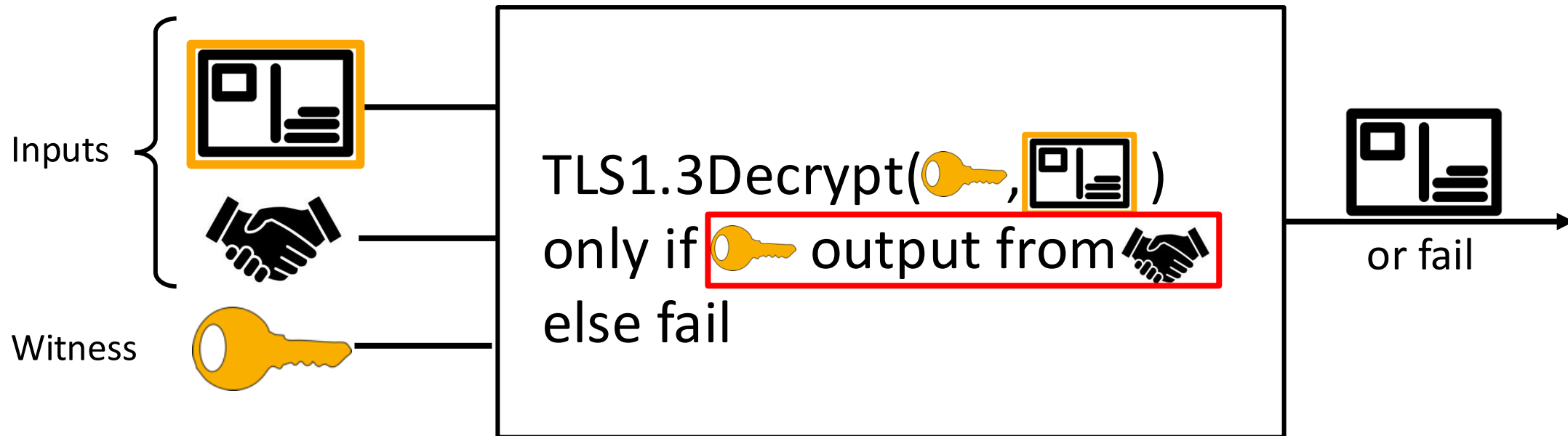
Idea to fix: client must prove key
was handshake output.



Channel Opening for TLS 1.3

How to open a TLS 1.3 ciphertext?

Idea to fix: client must prove key
was handshake output.



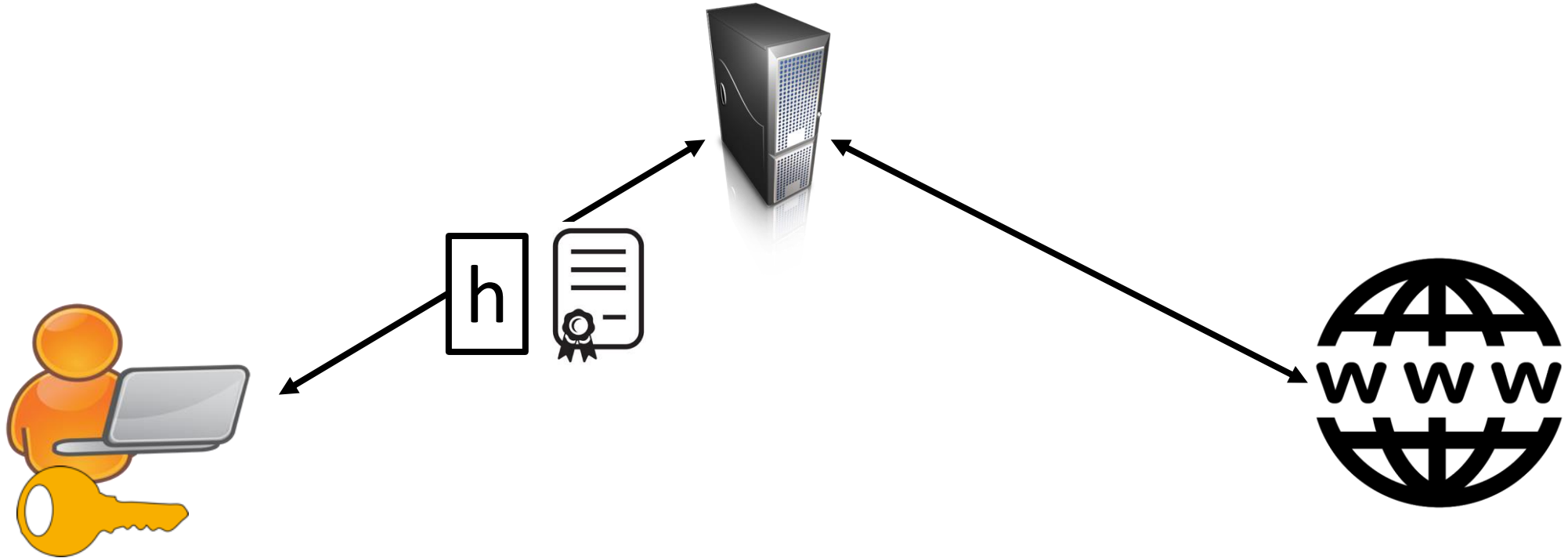
Key Consistency Check for TLS 1.3

(the short version)

- Simple, inefficient: re-run most of client's key derivation in circuit.
 - Diffie-Hellman values are binding to shared secret.
- Observation: handshake “commits to” intermediate steps of key derivation. Check these to shortcut key derivation.

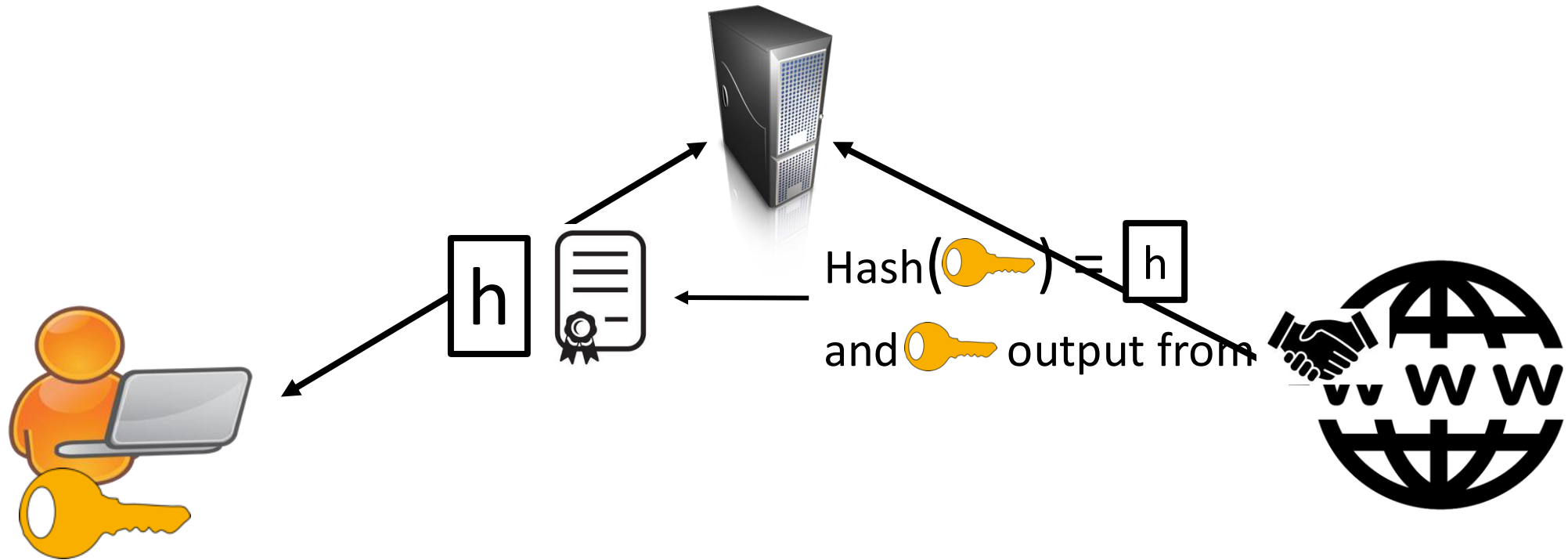
Amortizing the check

Key consistency check can be done once per session; later proofs can re-use.



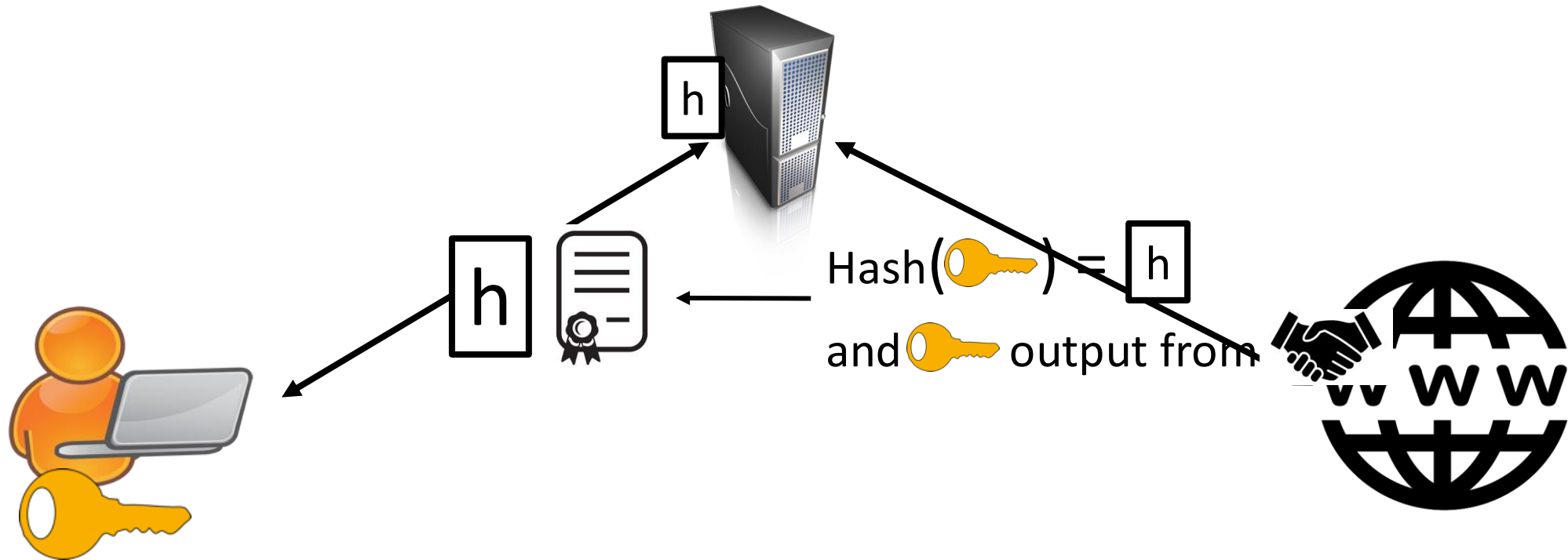
Amortizing the check

Key consistency check can be done once per session; later proofs can re-use.

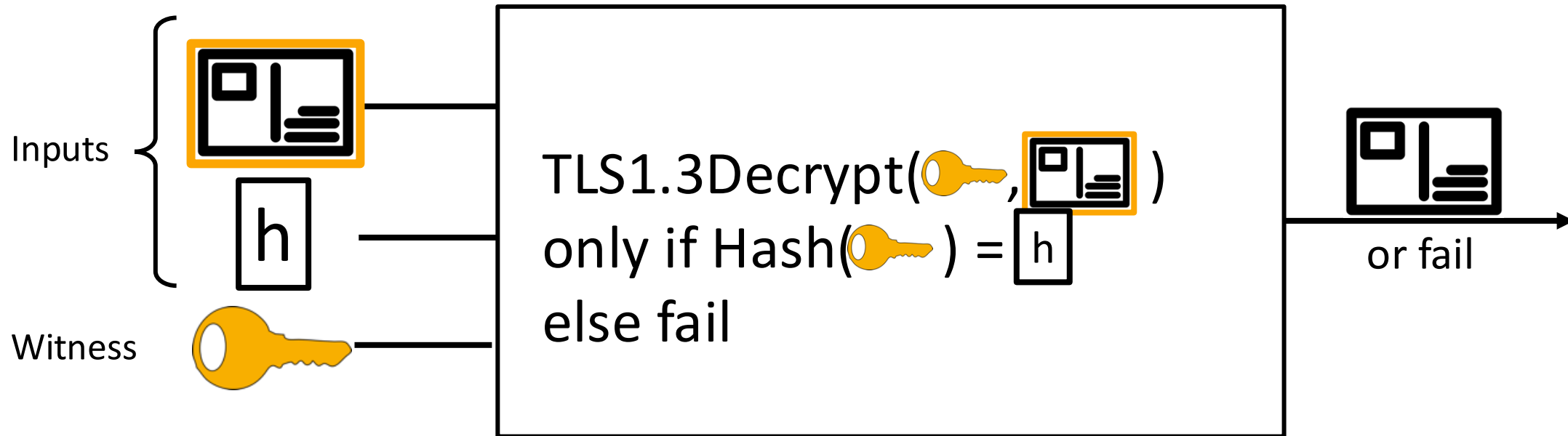


Amortizing the check

Key consistency check can be done once per session; later proofs can re-use.

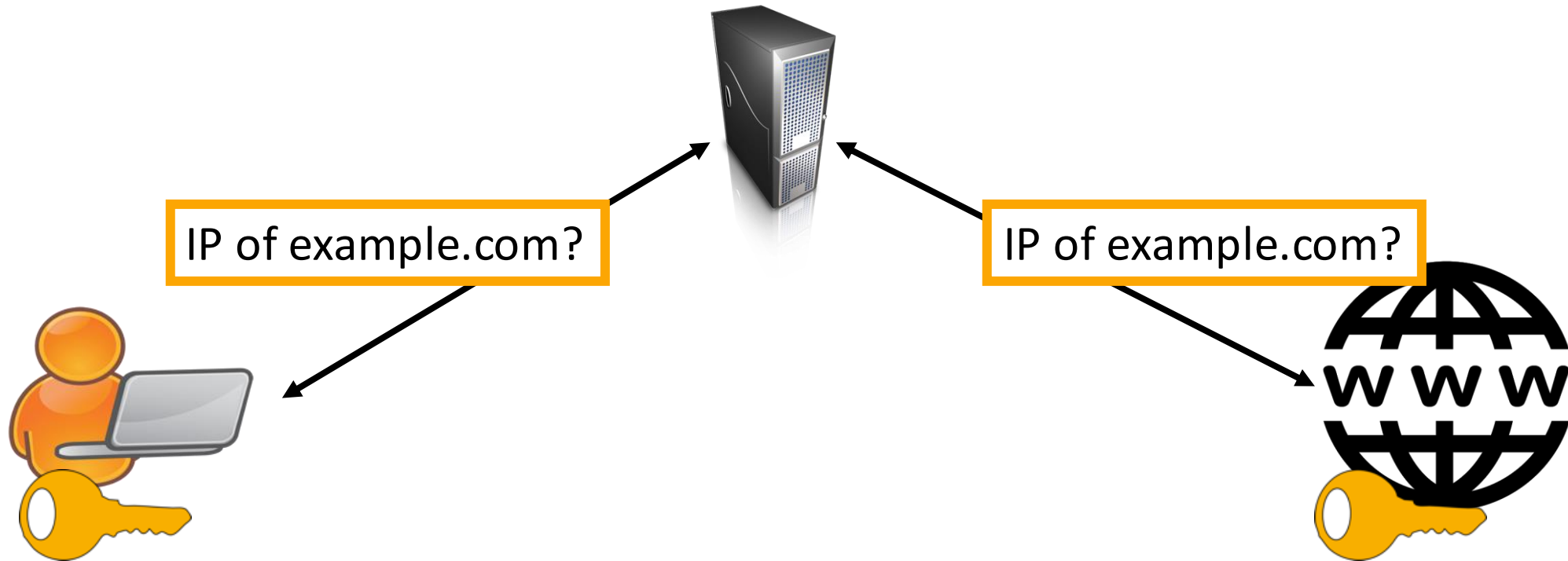


Amortizing the check



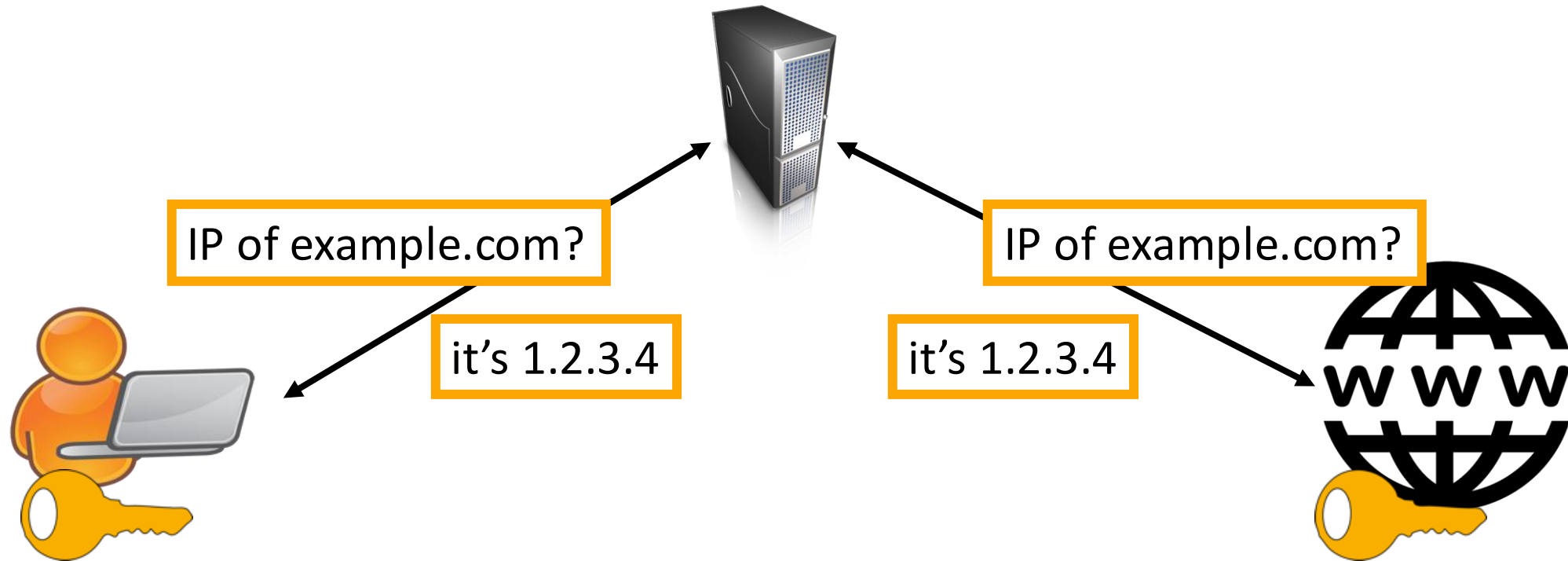
Encrypted DNS

DNS-over-{HTTPS, TLS}: DNS queries sent to a trusted resolver via TLS 1.3. Bypass local network's resolver.



Encrypted DNS

DNS-over-{HTTPS, TLS}: DNS queries sent to a trusted resolver via TLS 1.3. Bypass local network's resolver.



Encrypted DNS

DNS-over-{HTTPS, TLS}: DNS queries sent to a trusted resolver via TLS 1.3. Bypass local network's resolver.



Enabled by default in
Firefox, Chrome, Edge



IP of example.com?

it's 1.2.3.4



IP of example.com?

it's 1.2.3.4



Encrypted DNS

DNS-over-{HTTPS, TLS}: DNS queries sent to a trusted resolver via TLS 1.3. Bypass local network's resolver.

By design, local network can't see client DNS traffic - can't enforce filtering policy!



Enabled by default in Firefox, Chrome, Edge



IP of example.com?

it's 1.2.3.4

Blocklist:

...
blocked.com
...



IP of example.com?

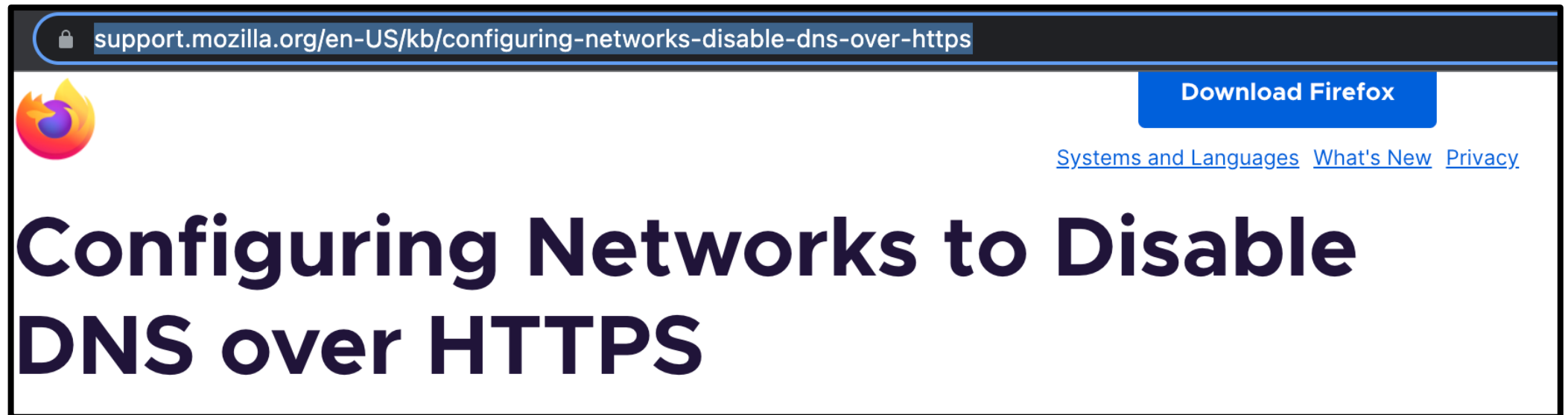
it's 1.2.3.4



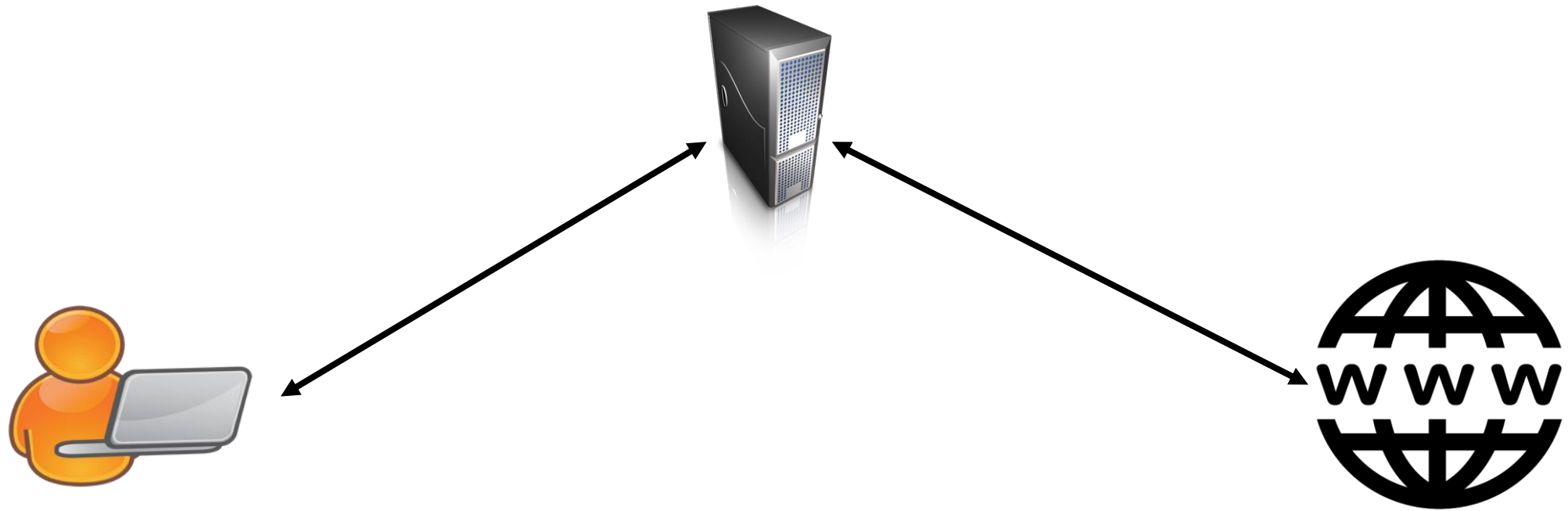
Children's Internet
Protection Law (2000)

TITLE XVII—CHILDREN'S INTERNET PROTECTION

The protection measures must block or filter Internet access



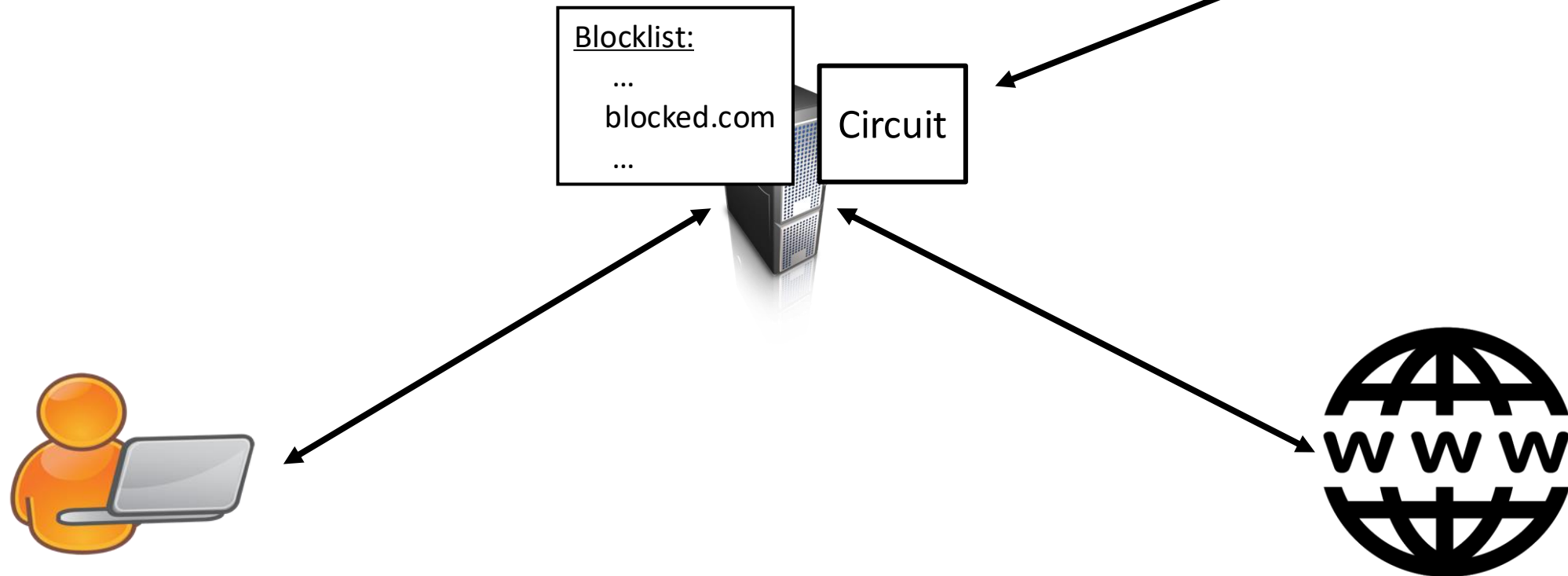
ZKMB for Filtering Encrypted DNS



ZKMB for Filtering Encrypted DNS

1. Network creates DNS blocklist and circuit

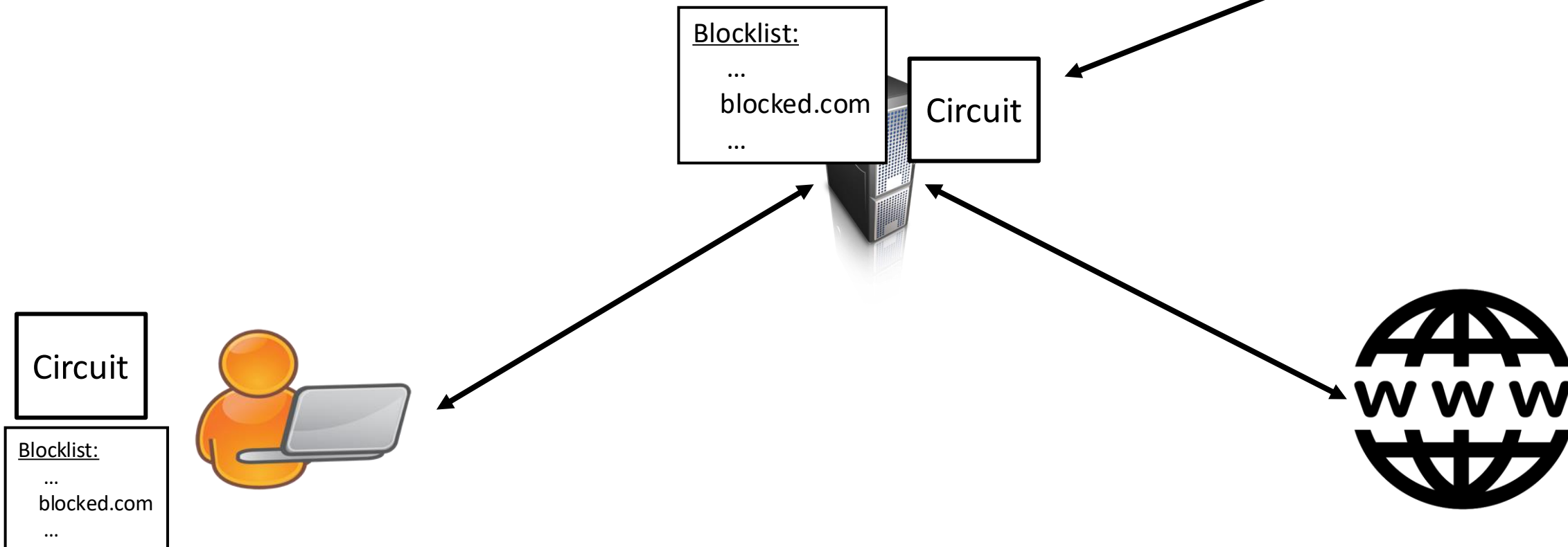
Channel opening: decrypt DNS query
Parse+Extract: deserialize domain name
Policy Check: verify set non-membership proof



ZKMB for Filtering Encrypted DNS

1. Network creates DNS blocklist and circuit
2. Clients get circuit + blocklist on network join

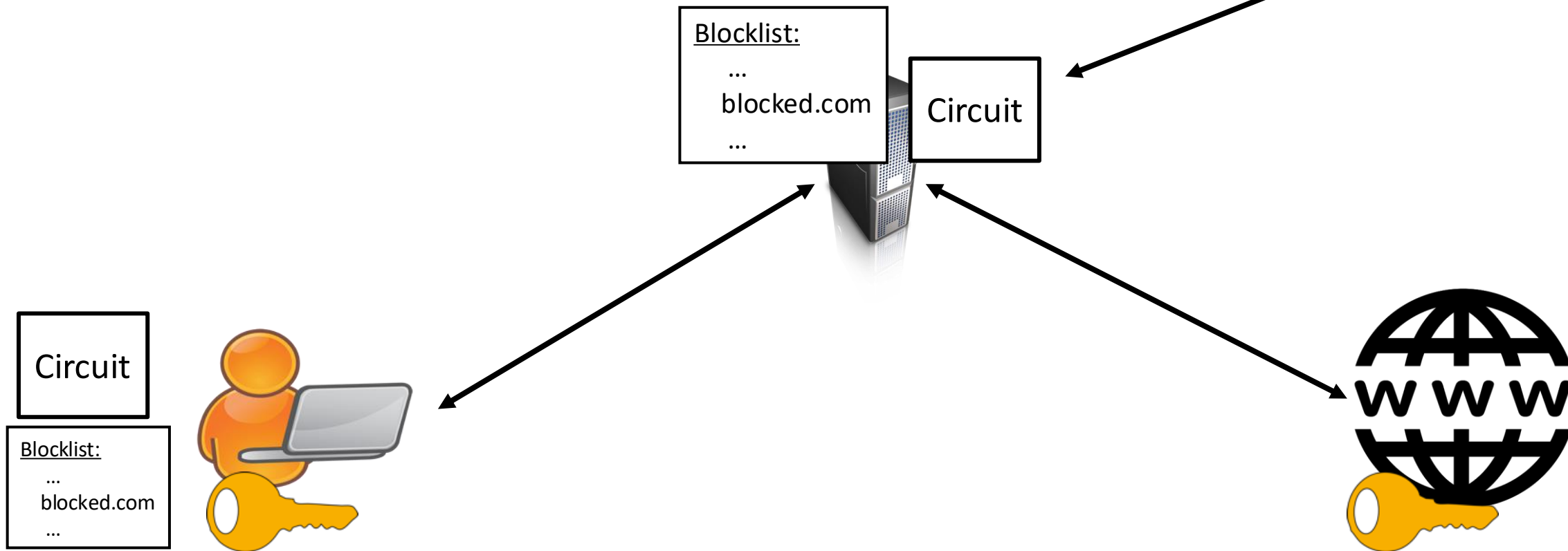
Channel opening: decrypt DNS query
Parse+Extract: deserialize domain name
Policy Check: verify set non-membership proof



ZKMB for Filtering Encrypted DNS

1. Network creates DNS blocklist and circuit
2. Clients get circuit + blocklist on network join
3. TLS1.3 handshake

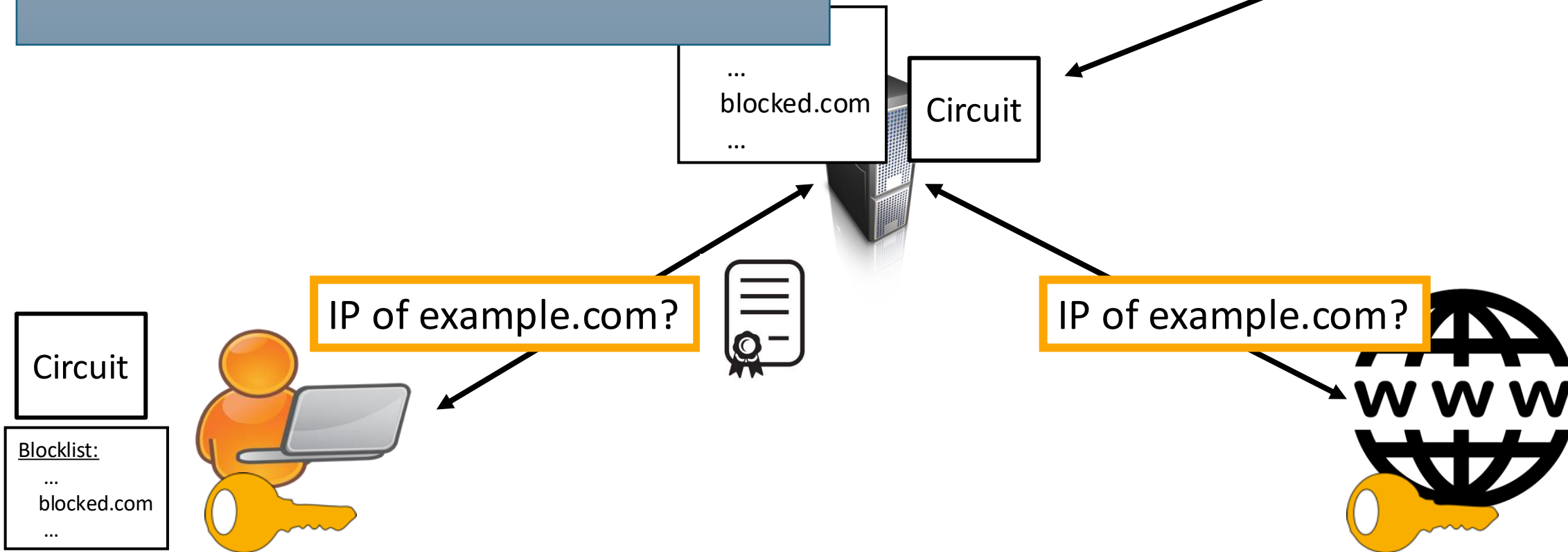
Channel opening: decrypt DNS query
Parse+Extract: deserialize domain name
Policy Check: verify set non-membership proof



ZKMB for Filtering Encrypted DNS

1. Network creates DNS blocklist and circuit
2. Clients get circuit + blocklist on network join
3. TLS1.3 handshake
4. Client sends query ciphertext + ZKP

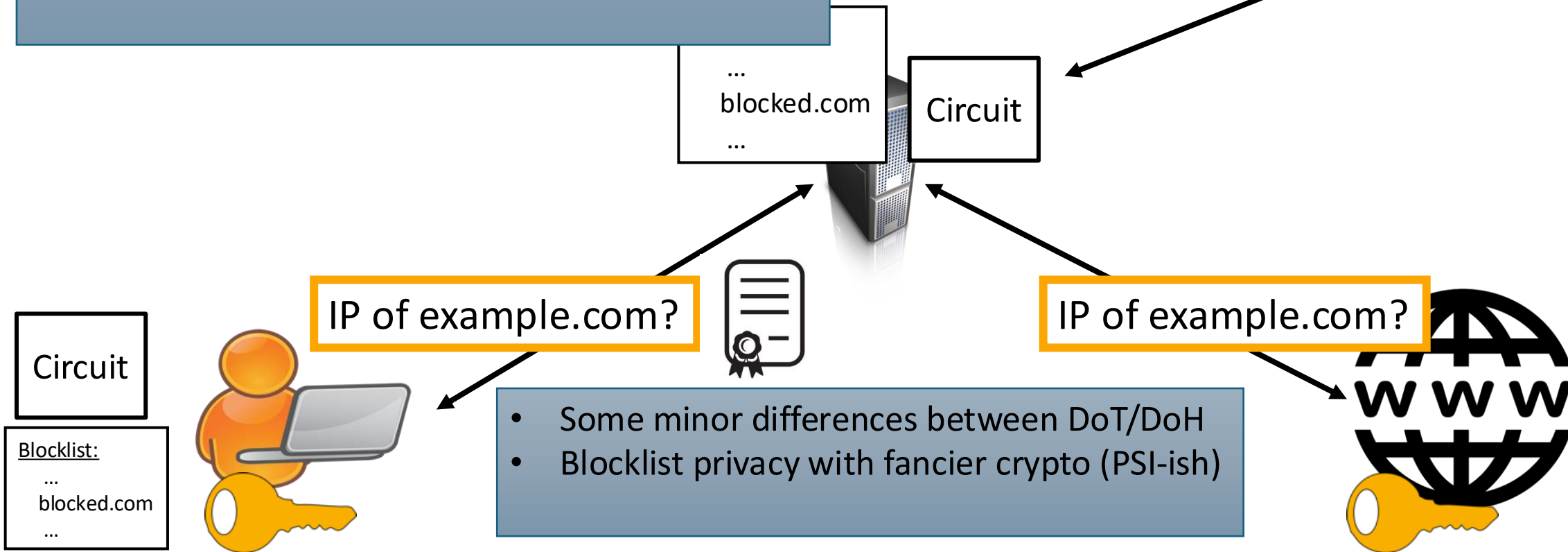
Channel opening: decrypt DNS query
Parse+Extract: deserialize domain name
Policy Check: verify set non-membership proof



ZKMB for Filtering Encrypted DNS

1. Network creates DNS blocklist and circuit
2. Clients get circuit + blocklist on network join
3. TLS1.3 handshake
4. Client sends query ciphertext + ZKP

Channel opening: decrypt DNS query
Parse+Extract: deserialize domain name
Policy Check: verify set non-membership proof



Experimental Results

Groth16 ZKP, 8 threads

Key Consistency Proof (once-per-session)

Method	#Gates (mil)	Prv time (s)	SRS (MB)	Proof size (b)	Vf time (ms)
Baseline	7.5	94.0	1200	128	~5
Optimized	1.1	16.5	149	128	~5

DNS Case Studies (excluding once-per-session setup)

Case Study	Ctxt size	#Gates (k)	Prv time (s)	SRS (MB)	Proof size (b)	Vf time (ms)
DoH (AES)	500	495	6.8	75	128	~5
DoT (ChaCha)	255	195	3.1	32	128	~5

Experimental Results

Groth16 ZKP, 8 threads

Key Consistency Proof (once-per-session)

Method	#Gates (mil)	Prv time (s)	SRS (MB)	Proof size (b)	Vf time (ms)
Baseline	7x 7.5	6x 94.0	8x 1200	128	~5
Optimized	1.1	16.5	149	128	~5

DNS Case Studies (excluding once-per-session setup)

Case Study	Ctxt size	#Gates (k)	Prv time (s)	SRS (MB)	Proof size (b)	Vf time (ms)
DoH (AES)	500	495	6.8	75	128	~5
DoT (ChaCha)	255	195	3.1	32	128	~5

Experimental Results

Groth16 ZKP, 8 threads

Key Consistency Proof (once-per-session)

Method	#Gates (mil)	Prv time (s)	SRS (MB)	Proof size (b)	Vf time (ms)
Baseline	7.5	94.0	1200	128	~5
Optimized	1.1	16.5	149	128	~5

DNS Case Studies (excluding once-per-session setup)

Case Study	Ctxt size	#Gates (k)	Prv time (s)	SRS (MB)	Proof size (b)	Vf time (ms)
DoH (AES)	500	495	6.8	75	128	~5
DoT (ChaCha)	255	195	3.1	32	128	~5

Prototype can generate proof for nontrivial ZKMB in 3.1 seconds.

Agenda for this lecture

- Announcements
- Zero-knowledge middleboxes (motivation)
- Zero-knowledge middleboxes for DNS filtering
- zkTLS: the problem it solves

“zkTLS” and TLS oracles

- ZKMBs solve the problem of *policy enforcement* on traffic going out of a network.
- zkTLS solves a complementary problem: allowing client to assert they received some data from a web server.
 - Example: I want to join a stock trading platform but I need to convince them that I have >5k in my bank account. Only my bank can say authoritatively how much is in my account.
- Interesting twist: do it without inline third party. Requires multiparty computation.
- Privacy not always needed in TLS oracles and zkTLS: main point is preventing forgery of messages. ZKP can be layered on top

Question: how to solve this with no legacy constraints?